

TESTING MULTIPATH TCP AND CONGESTION CONTROLS ON THE LINUX 6.8 KERNEL IN A PROXMOX VIRTUAL ENVIRONMENT

By

Corbin Forrester

May, 2025

Director of Thesis: Dr. John L. Pickard

Department of Technology Systems

ABSTRACT

This thesis investigates the performance of Multipath TCP (MPTCP) in Linux Kernel 6.8 using a low-cost Proxmox virtual environment. Key findings reveal that MPTCP in Linux Kernel 6.8 is significantly influenced by single-path congestion control configurations, allowing for better optimization for wireless networks than expected from specifications in RFC 8685. The research identifies optimal congestion control settings for MPTCP in a simulated 5G and Wi-Fi 6 environment: BBR and BIC for maximizing bandwidth, Westwood+ for ensuring fairness, and Vegas for low-priority, low-latency flows. While MPTCP achieved 1.125-1.4 times more bandwidth than a competing single-path TCP flow, it maintained high fairness as measured by Jain's Fairness Index. Additionally, the study finds that TCP-LP, designed for low-priority traffic, is dysfunctional in Linux Kernel 6.8, indicating a need for kernel updates. These findings provide actionable guidance to system administrators and application developers seeking to optimize network utilization, particularly for mobile devices with dual connectivity. Moreover, they have implications for emerging protocols like QUIC and MPQUIC, which share similar congestion control mechanisms with TCP and MPTCP, and the future of the HTTP/3 internet.

TESTING MULTIPATH TCP AND CONGESTION CONTROLS ON THE LINUX 6.8
KERNEL IN A PROXMOX VIRTUAL ENVIRONMENT

A Thesis

Presented To the Faculty of the Department of Technology Systems

East Carolina University

In Partial Fulfillment of the Requirements for the Degree

Master of Science in Network Technology

By

Corbin Forrester

May 2025

Director of Thesis: John L. Pickard, Ph.D

Thesis Committee Members:

Ciprian Popoviciu, Ph.D

Peng Li, Ph.D

© 2025, Corbin Forrester

Tables of Contents

List of Tables	v
List of Figures.....	vi
Abbreviations	vii
Chapter One: Introduction	1
Research Background.....	1
<i>QUIC, MPQUIC, and HTTP/3.....</i>	<i>5</i>
Research Problem.....	6
Research Questions	7
Research Objectives	8
Significance of the Study	8
Assumptions	9
Limitations	10
Chapter Two: Literature Review	12
Standard TCP CC	12
High Speed and Long Delay TCP Congestion Control.....	15
Random and Link Loss TCP Congestion Control.....	20
Satellite Network Designed Congestion Control	23
Other TCP Congestion Controls	24
MPTCP	25
<i>Coupled Congestion Control.....</i>	<i>26</i>
<i>Schedulers.....</i>	<i>32</i>
QUIC33	
<i>MPQUIC.....</i>	<i>36</i>
Chapter Three: Methodology	38
Design.....	38
The Following Tests Were Performed:	39
Chapter Four: Results	41

Is MPTCP Using Uncoupled Congestion Control?.....	41
Is MPTCP Fair?.....	41
TCP LP for Bandwidth Without Interrupting Other Flows.....	43
Testing Bitrate With Random Packet Loss With CUBIC, BBR, Illinois, And YeAH .	44
Test Link-Loss Tolerant Congestion Control Against a Competing CUBIC flow.	45
The Best MPTCP Settings in a Wireless Network.....	47
Chapter Five: Conclusions	53
References.....	56
Appendix A: Proxmox Install	63
Appendix B: Network Setup	64
Linux Shapers Setup.....	64
Appendix C: Traffic Shaper Commands.....	71
Appendix D: Linux VM Set Up	72
Appendix E: MPTCP Settings	73
Appendix F: iperf3 Commands	74

List of Tables

Table 1 Maximum Bandwidth Available with Netem Delay	10
Table 2 CUBIC, BBR, Illinois, and YeAh Bitrate Results with Random Link Loss	44
Table 3 Competing Congestion Controls Over a Random Packet Loss Bottleneck Link	46
Table 4 MPTCP Bitrate on a Congested Network.....	48
Table 5 Single-Path TCP Bitrate over a Congested Network.....	49
Table 6 Improvement of MPTCP Over SPTCP in a Congested Network.....	50

List of Figures

Figure 1 MPTCP in Proxmox Network Diagram	38
Figure A1 <code>cat /etc/rc.local</code> on Shaper8	67
Figure A2 <code>ip route show</code> on Shaper8	67
Figure A3 <code>ip route show</code> on Shaper9	68
Figure A4 <code>ip route show</code> on Shaper7	68
Figure A5 <code>cat /etc/network/interfaces</code> on Shaper8	69
Figure A6 <code>cat /etc/network/interfaces</code> on Shaper9	69
Figure A7 <code>cat /etc/network/interfaces</code> on Shaper7	70

Abbreviations

BDP	Bandwidth Delay Product
CWND	Congestion Window
HoL	Head of Line
MPQUIC	Multi-Path Quick
MPTCP	Multi-Path Transmission Control Protocol
MSS	Maximum Segment Size
QoE	Quality of Experience
QUIC	Quick UDP Internet Connections
RTO	Retransmission Timeout
RTT	Round-Trip Time
RWND	Receive Window
SPTCP	Single Path Transmission Control Protocol
SSThresh	Slow Start Threshold
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UDP	User Datagram Protocol

Chapter One: Introduction

Research Background

TCP (transmission control protocol) has been the backbone of the internet since its introduction in RFC 793 (Postel, 1981). It was then compiled and updated in RFC 9293 (Eddy, 2022). RFC 9293 obsoletes RFCs 793, 879, 2873, 6093, 6429, 6528, and 6691 and updates 1011, 1122, and 5961. The original TCP would establish a connection with a three-way handshake using a SYN flag, a SYN-ACK flag, and an ACK flag, where each endpoint would establish its receive window (RWND). Sequence numbers would be assigned to each data octet sent over the connection, and reliability in the connection is established by the ACK sent back on the connection, including the last sequence number received. These sequence numbers are decided randomly at the start of the connection at the TCP handshake. By using the sequence numbers, connections can correctly order and verify what data has been sent. The receive window would dictate how much data can be on the network before an acknowledgment is sent by the receiver, so that the receiver will not need to drop packets due to its buffer being filled. Each RWND received would return an ACK to the sender, creating a sending rate of $RWND/RTT$ (round-trip time) bytes per second. This delivery method assumed the delivery rate would be limited by bottlenecks on the path between endpoints and the receiver's RWND. If no acknowledgment is received within a timeout period, called retransmission timeout (RTO), the packets are considered lost and retransmitted.

Within a few years, the first congestion collapse occurred in October 1986. The data connection from sites merely 400 yards apart dropped from 32 kb/s to 40 b/s on a campus network. This network had dropped to .1% efficiency (Jacobson, 1988). Without congestion control, too many devices on the network would try to send their RWND worth of data, filling

the router's buffers between the connections, which would drop the excess packets. When packet loss was detected with an RTO, the endpoints would again send an RWND worth of data on the wires. This led to all connections not being reliably transmitted; there was a failure in only accounting for the receiving computer's buffer and not the router's buffer on the path to the receiver.

To correct this problem, the first congestion control algorithm was proposed and introduced, described in Van Jacobson's "Congestion Avoidance and Control" and later known as TCP-Tahoe (Jacobson, 1988). An essential feature of congestion control is fairness to other TCP flows to prevent collapse. An updated version of TCP-Tahoe was introduced as a standard in RFC 1122, known as TCP-Reno (Braden, 1989). TCP-Reno became the dominant TCP congestion control on the internet for over a decade until it was determined not to be efficient enough to handle traffic on high-speed, high-latency links known as "long and fat" networks. Many proposed replacements were created, and eventually, CUBIC-TCP became the default successor to TCP-RENO (Ha et al., 2008). CUBIC had better bandwidth usage over long networks and had decent fairness to other TCP streams. It was also fairer to flows with different RTT times, which was an issue with TCP-Reno and many high-speed replacement proposals (Xu et al., 2004).

TCP-Reno and CUBIC-TCP do not have mechanisms to differentiate packet loss caused by congestion versus packet loss caused by wireless radio link loss. When TCP-Reno was envisioned, computer networks were entirely wired, and packet loss on wired networks could be assumed to be 99% caused by congestion on router buffers (Jacobson, 1988). When wireless networks became widespread, many packet loss signals occurred due to poor reception at radio links. Many congestion protocols were proposed to differentiate between congestion packet loss

and radio link packet loss. The most common methods are to rely on explicit congestion notifications (ECN) and measuring changes in RTT to determine when buffers were being filled on routers on the connection. The most powerful contender for lossy-link tolerant congestion control has been Google’s BBR (bottleneck bandwidth and round-trip propagation time) congestion control (Cardwell et al., 2017).

Many servers and computers will often have multiple network interfaces, but the TCP protocol is limited to using one IP address and one path. Taking advantage of multiple network interfaces usually requires link aggregation, which combines the network interfaces to act as one interface, increasing bandwidth and reliability. Other devices do not have symmetry in network interfaces to link aggregate, which requires administrative planning and effort to configure manually. Cellphones often have access to Wi-Fi networks and cellular networks simultaneously, but can only use one network interface per TCP flow. Multipath TCP (MPTCP) is a TCP extension that allows multipath connections, which is impossible for standard TCP. The original RFC for MPTCP version 0 was RFC 6824 (Ford et al., 2013), and it has been superseded by version 1 in RFC 8684 (Ford et al., 2020). It requires kernel support and, as such, has mostly been deployed in controlled client-to-server situations. Apple iOS has supported MPTCP since iOS 7 and has deployed it in applications such as Siri and, since iOS 13, Apple Maps and Apple Music. Linux Kernel MPTCP support has been available since Linux kernel 5.6, using MPTCP version 1, and is enabled by default (MPTCPDevTeam, 2024). In order to implement MPTCP, the application needs to be programmed to use it or be forced into using MPTCP with another application, like `mptcpize`, which is used in this experiment to make `iperf3` MPTCP capable.

Using multipath TCP can create unfairness, where a single MPTCP connection can use multiple single-path TCP connections, taking a larger share of bandwidth than what should be

fair. For example, a three-path network card connection will use the same bandwidth that three single-path TCP flows would have on their own. This led to the development of coupled congestion controls, such as LIA, Link Increase Aggregation (Raiciu et al., 2011), which should be the default MPTCP congestion control according to MPTCP version 1 (Ford et al., 2020). LIA uses a modified TCP-Reno with decreased CWND (congestion window) growth on each path to maintain fairness with competing TCP flows. There is ample room for improvement, as LIA was created well after CUBIC-TCP was already the default congestion control, meaning MPTCP may not be as efficient on long and fat networks. Being similar to TCP-Reno also means MPTCP may not be efficient in determining radio link loss and loss bandwidth on wireless networks. Considering MPTCP is mainly used on cellphones, this can be a significant inefficiency. Even though LIA should be the default congestion control for MPTCP according to the RFC standard, the Linux kernel does not appear to have LIA as a congestion control option, and MPTCP does respond to changing the traditional congestion controls on Linux. This was evaluated and demonstrated in this experiment.

MPTCP relies on a scheduler to decide which data segments are sent over which path. The default scheduler for MPTCP is min RTT, which fills the congestion window with the lowest RTT before sending traffic to other subflows (Vu et al., 2022). Other schedulers include Round Robin, which always alternates paths for the next data sent, and Redundant, which repeats the traffic on all paths for reliable transmission (possibly useful in Ultra-Reliable Low Latency 5G communications); heuristic-based schedulers such as blocking estimation-base scheduler, BLEST, and Earliest Completion First, ECF; and learning-base schedulers like Peekaboo and ReLeS using deep reinforcement learning. These schedulers, along with the congestion control, have an influential impact on the performance and behavior of MPTCP. Despite these options, no

scheduler option is available in the Linux Kernel, and it is assumed that only minRTT is used by default.

QUIC, MPQUIC, and HTTP/3

The most recent version of HTTP, version 3, was officially standardized in RFC 9114 in 2022. This new protocol stack includes using QUIC protocol instead of TCP for reliable internet communication. One reason for doing so is that TCP multiplexing does not modify the TCP packet loss recovery mechanisms, meaning that all streams get interrupted by the packet loss in a single stream. QUIC does not suffer from this drawback, having reliability at the stream level and congestion control across the entire connection. Most stream management happens at the QUIC layer and not HTTP/3, unlike in HTTP/2, where HTTP/2 handles multiplexing and not the TCP protocol (Bishop, 2022).

Some early tests of HTTP/3 over HTTP/2 proved that its performance benefits were isolated to scenarios with high latency or poor bandwidth. Later studies suggested that benefits are more pronounced in high-latency scenarios and mobile networks, but no improvement in use cases like video streaming (Sengupta et al., 2024; Trevisan et al., 2021). Ultimately, security is better as HTTP/3 requires HTTPS and TLS (transport layer security) version 1.3 or better. Regarding configuration, a TCP connection with a more efficient congestion control and settings may account for better performance than a poorly configured QUIC protocol with a less efficient congestion control and settings (Chaudhary et al., 2024).

QUIC is an application-layer protocol that leverages UDP (User Datagram Protocol) instead of TCP on the fourth layer. The primary difference between TCP and UDP is reliable, verified service delivery, which includes congestion control, which is absent in UDP. QUIC uses congestion control on a different layer than TCP and is considered by middleboxes to be a UDP

connection. It has enhanced security and performance over regular TCP and can be more easily deployed as it exists on the application layer rather than the transport layer. This means it can be deployed and updated without interfering with the operating system kernel, as TCP and MPTCP must (Iyengar & Thomson, 2021). For example, Google can update its Chrome browser to deploy a new or updated version of QUIC instead of an entire OS update. A multipath version, MPQUIC, is currently in internet draft (Viernickel et al., 2018). Despite being a new protocol, QUIC and MPQUIC use the same or nearly the same congestion controls and schedulers as TCP and MPTCP.

The findings in this study can be applied functionally to QUIC and MPQUIC in similar network situations. TCP connections can be interchanged for QUIC connections, and MPTCP can be interchanged for MPQUIC connections. Because QUIC and MPQUIC do not need an OS update to add features, they should be more easily updated and modified to include different congestion controls and schedulers. Many of the performance advantages of using well-tuned congestion controls and schedulers also have the greatest impact the worse the network situation is, meaning a highly lossy link with high RTT will see the most performance gained from a well-tuned HTTP/3 stack and that a well-tuned TCP or MPTCP connection will outperform a poorly configured QUIC or MPQUIC connection. MPQUIC and QUIC connections can fall back on MPTCP and TCP if QUIC is unavailable.

Research Problem

Despite showing improvements over single-path TCP connections in bandwidth, reliability, and network resource utilization, particularly in use with mobile devices with dual connectivity with 5G and Wi-Fi 6, MPTCP has not been widely deployed. Using well-configured MPTCP in smartphone applications would increase network utilization, efficiency, and quality of

user experience. Although MPTCP is enabled by default in the Linux Kernel, it does not have many of the options for MPTCP that would be expected after reading the literature and RFC standards, including coupled congestion controls and schedulers. This gap is due to researchers relying on theoretical simulations to improve the multipath protocol rather than testing the protocol as deployed on operating systems. Furthermore, there is limited public documentation on how to functionally tune MPTCP in the Linux Kernel or determine efficient configurations for different use case scenarios.

The primary use case explored in this laboratory setup is the best congestion control to use, in terms of throughput, in the delivery of service to a mobile device. This study presents an accessible hardware setup laboratory to verify claims for congestion controls and test the current deployment of MPTCP in the Linux Kernel 6.8. These results can also help direct the decisions on QUIC and MPQUIC congestion controls and schedulers, as HTTP/3 being poorly configured can cost performance. The single-path congestion controls available in the Linux Kernel will also be tested. This research aims to enhance the quality of service for mobile users, particularly in regions with limited network infrastructure, and contribute to the development of more efficient and reliable multipath protocols.

Research Questions

1. To what extent does the implementation of MPTCP in the Linux Kernel 6.8 comply with the standards set by RFC 8684, particularly in the use of coupled congestion control?
2. How is MPTCP in Linux Kernel 6.8 affected by setting the single-path congestion controls?

3. Which congestion control algorithms optimize MPTCP to send data to a mobile device with dual connectivity in terms of bandwidth utilization and fairness?
4. To what extent does TCP-LP successfully identify and utilize available bandwidth without disrupting competing TCP flows?

Research Objectives

1. Test the influence of uncoupled congestion control algorithm selection on MPTCP performance in terms of throughput and fairness.
2. Develop an accessible and affordable network laboratory using Proxmox that can simulate real-world network conditions, enabling the testing of MPTCP and congestion control algorithms on Linux Kernel 6.8.
3. Recommend updates and fixes to the currently deployed Linux congestion controls.
4. Provide actionable recommendations for optimizing MPTCP deployment in Linux Kernel 6.8, particularly in delivering service to mobile devices with dual connectivity with 5G and Wi-Fi 6.

Significance of the Study

The practical significance of this research lies in its potential to enhance the quality of service for mobile users, particularly in regions with limited network infrastructure. By testing and verifying the performance of MPTCP as deployed in Linux Kernel 6.8, this study evaluates the best configurations for optimizing MPTCP in mobile environments. Most academic research on MPTCP does not use the active deployment of MPTCP version one in the Linux Kernel, failing to recreate the current real-world deployment of the protocol available for most developers. Using the real-world deployment of the protocol and operating systems, its findings

directly apply to current application deployments. This research supports economic and social development in global deployments, where in areas with weaker infrastructure, better reliability, and efficient network performance are invaluable.

Furthermore, this study has broader implications for the future of multipath protocols. MPQUIC may be standardized and commonly deployed in the future, and congestion control and scheduler tuning may be important optimization features. Many congestion controls tested in this research and multipath schedulers will be interchangeable in practical deployments between MPTCP and MPQUIC. MPTCP is currently deployed on Linux and activated by default, meaning that most Android smartphones and iPhones can accept an MPTCP connection if an application developer chooses to implement it. This presents an opportunity for real-world deployments to test, refine, and improve network resource utilization using multipath protocols, which may become standard for mobile device service delivery in the future.

Assumptions

Assumptions include that this virtual network laboratory will accurately reflect an actual hardware deployment. This includes the fact that the Debian routers will handle traffic in a similar manner to real-world routers on the internet and that the virtual switches and the Debian router's traffic shaping can accurately reflect real-world wireless and wired data networks. Another assumption is that the application "mptcpize" accurately recreates the behavior an application programmed to use MPTCP would display. "mptcpize" forces the application to load MPTCP sockets instead of TCP sockets (MPTCP Dev Team, 2024).

According to the Linux MPTCP Upstream Project (2024), no significant changes have been made to MPTCP in the Linux Kernel from 6.8 to 6.12 as of the current writing of this

experiment. It is assumed that this deployment of the Linux Kernel using MPTCP reflects a real-world environment.

Limitations

Limitations include costs such as being unable to use real-world routers to run the tests and not having deployed this same hypervisor and virtual network on different hardware configurations. Additionally, testing has shown that the delay function reduces the maximum bandwidth in an exponential function, as shown in the results in Table 1. As the artificial delay increases, the maximum bandwidth reported by the `iperf3` command decreases. It is currently unknown why this occurs, and it does not seem to change depending on the congestion algorithm selected. It is possible this is a limitation of the hardware, but comparing the hardware was out of the scope of this experiment. The RTT unfairness tests also failed to show any difference between competing RTT flows as expected. This lab appears to be unable to reproduce RTT unfairness.

Table 1

Maximum Bandwidth Available with Netem Delay

Netem delay in ms	Bandwidth in Mbit/s
0	11300
5	4000
10	2700
15	1800
20	1250
40	669
80	323
160	175
320	73.3
640	34.4
1280	11.2
2560	3.36

Note. As network-emulated delay increases, the maximum bandwidth in the lab decreases.

This limits the use of delay in testing the experiments, which is relevant to the minRTT scheduler used in MPTCP. For example, to see minRTT not using a path, the total expected bandwidth cannot exceed the amount that the RTT allows for.

The expectation is that with only one scheduler available, the minRTT, the congestion control that can best take advantage of this, and being wireless radio link loss tolerant, will have the best performance. First, we test if the MPTCPv1 implementation changes with single-path TCP congestion control configurations. Then, we test if MPTCP acts fairly in relation to other flows. Then, we test which congestion control is best in a double wireless simulated environment.

The minRTT scheduler relies on filling buffers to send the next data on the path with the lower RTT, allowing for the use of both paths and limiting the head-of-line blocking. When using a delay-based congestion control, the single-path control will attempt to minimize the delay on both paths, leading to the majority of the flow staying on the path with the lowest RTT. Uneven RTT may even produce MPTCP that has lower performance than a single path.

Chapter Two: Literature Review

The available congestion controls in the 6.8 Linux kernel are as follows: Bottleneck Bandwidth and RTT (BBRv1), Binary Increase Congestion control (BIC), CAIA Delay-Gradient (CDG), CUBIC, Data Center TCP (DCTCP), High-Speed TCP, H-TCP, TCP Hybla, TCP Low Priority (TCP LP), TCP Illinois, TCP-NV (New Vegas), Scalable TCP, TCP Vegas, TCP Veno, TCP Westwood+, and YeAH TCP. These are found in the root/net/ipv4 section of the Linux kernel (Linux Kernel Development, 2024). None of these congestion control algorithms are exclusive to TCP. TCP congestion control is divided into five sections: standard, high speed and/or long delay, radio link-loss tolerant, satellite network designed congestion control, and other.

Standard TCP CC

Standard TCP congestion control is often based on the original TCP-Tahoe congestion control and later the common TCP-Reno. Most implementations will always assume that packet loss is due to congestion.

The original TCP-Tahoe from Van Jacobson's paper "Congestion Avoidance and Control" (1989) initially created a slow start algorithm (which is not very "slow") and a congestion avoidance algorithm. A slow start would increase each MSS (maximum segment size) sent before receiving an ACK by a factor of 2, as in 1, 2, 4, 8, 16, etc., until it hits congestion or the slow start threshold (SSthresh). Once the stream hits congestion, avoidance would occur. The SSthresh would be reduced by half, CWND would be reduced to 1, the connection rate would start at 1 MSS, and slow start would begin again until hitting the new SSthresh, a receive window, or congestion again. Upon hitting the SSthresh, congestion avoidance would occur where the TCP sender would only increase the packets sent by 1 MSS for each ACK until hitting

congestion or the receive window. This was sufficient to avoid congestion collapse. Congestion was detected by the then-novel approach of detecting lost packets as the reason for congestion instead of modifying TCP with new parameters to detect congestion.

This belief that dropped packets indicates congestion comes from the fact that in a wired network, 99% of the time, dropped packets are due to congestion. So, when the RTO timed out, there was a 99% chance that the lost packets were due to congestion, so congestion avoidance should occur. Without congestion avoidance, upon detecting lost packets, the TCP protocol would try to send all of the packets considered lost with an entire receive window, again flooding the network with packets. As each device on the network detects loss and floods the network with its receive window worth of packets, the congestion continues, grinding the entire network to a near halt. TCP-Tahoe was the first implementation of congestion control for networks.

TCP-Reno began in 1990 as an improvement to TCP-Tahoe and was adopted as a standard. It was the most deployed congestion control algorithm until it was replaced by BIC and then CUBIC (Xu et al., 2023). TCP-Reno begins with the same slow start as TCP-Tahoe and then begins to optimize congestion control when detecting congestion, reducing both the SStresh and CWND by half and starting to send packets from that level in congestion avoidance instead of back at 1 MSS and using a slow start again. Reno will increase each set of packets sent by one MSS, starting at SStresh, until congestion is found and then reduce the SStresh by half. This creates a sawtooth pattern. This was sufficient for networks until the bandwidth capacity became so large that a TCP connection can take tens of seconds to tens of minutes in a linear progression to find the congestion window. TCP-Reno can be activated after receiving three duplicate ACKs from the receiving computer instead of waiting for an RTO, further optimizing congestion control.

A duplicate ACK is sent when an out-of-order segment arrives. When three duplicate ACKs are received, TCP can begin a "fast re-transmit," where the sender attempts to repair the TCP connection by sending the missing segment (Allman et al., 2009). "Fast recovery" is the algorithm that controls how to transmit data until duplicate ACKs stop arriving. TCP-Reno has a fault where if multiple packets are lost in one window, those duplicate ACKs can build up and decrease the cwnd by 2, 4, 8, etc., times before repairing the connection (Stoica, 2005). This means that in some cases, TCP-Reno performs just as poorly as TCP-Tahoe would.

TCP-NewReno from 1999 is a modification to TCP-Reno, which allows TCP to detect multiple packet loss. Unlike TCP-Reno, TCP NewReno does not leave Fast-Recovery until all the data transmitted since fast-recovery began is acknowledged. This issue caused TCP Reno to reduce its CWND multiple times from one congestion event, and NewReno corrected this behavior (Stoica, 2005).

Due to its additive increase design, TCP-Reno is RTT unfair, where flows with shorter RTTs can increase their connection speed faster than connects with longer RTTS, consuming more available bandwidth than what should be fair. This unfairness was noted in the TCP-BIC paper (Xu et al., 2004) with the equation of $(RTT2/RTT1)^2$.

TCP-Vegas, 1994, uses RTT to detect buffers between the receiver and sender to detect bandwidth and congestion. The lowest RTT is used to set the expected RTT, and the actual RTT is compared to this to detect congestion; if the actual RTT is much higher, congestion is assumed. This window is adjusted to every other RTT, where if congestion is detected, the CWIND is reduced by 1/8th. If congestion is not detected, the next segment sent will have one more MSS. Vegas retransmits data on the first duplicate ACK instead of waiting for 3 ACKS as in TCP-Reno. Other non-RTT detection CC algorithms can fill network buffers before congestion is

detected. The advantage of using RTT to detect congestion is that the congestion window adjusts before packet loss even occurs, and there can be lower latency on data being sent, which may be necessary to some programs that prefer low latency to bandwidth. TCP-Vegas keeps the same congestion recovery as TCP Reno to fall back on (Brakmo & Peterson, 1995). Because TCP-Vegas assumes congestion on the first duplicate ACK, it does not discriminate radio link loss and assumes any packet loss is congestion.

TCP-NV, New Vegas, is an update to Vegas with the goal of existing in data centers where low RTT time is valued and a controlled environment can prevent any competing loss-based congestion controls. It acts exactly as TCP-BIC until congestion is detected using RTT, similar to TCP-Vegas. RTT times with TCP-BIC in data centers showed a 50ms RTT, while NV remained under 1ms. Confusingly, other TCP-New Vegas schemes have been proposed, but the implementation in the Linux Kernel appears to be the data center version. Additionally, TCP-NV does not require router ECN support, unlike DCTCP (Brakmo, 2015).

High Speed and Long Delay TCP Congestion Control

While standard TCP worked well for a while, the additive increase to congestion control had started to be well outpaced by the bandwidth available for connections. Underutilized high bandwidth, large latency networks motivated the creation of better congestion controls to utilize the exponentially larger bandwidths available than when TCP-Reno was designed. Additionally, satellite networks have unique requirements where they can have large bandwidth available but have very high round trip times, making bandwidth and congestion detection difficult. Many of the congestion controls available in the 6.8 Linux Kernel are high-speed, long-delay-styled TCP congestion controls.

TCP-Highspeed (Floyd, 2003) (often abbreviated as HS-TCP) is an early attempt to modify TCP-Reno better to take bandwidth in long RTT high bandwidth links. Its RFC 3649 paper explains that a standard TCP connection with a 100 ms round trip and 10 Gb/s connection will take a congestion window of 83,333 segments, a packet drop rate of 1 out over every 5 billion packets, or one congestion event every hour and two-thirds, to recover from one congestion event. HS-TCP implements an exponential curve for CWND growth. By keeping the CWND growth the same for packet drops up to .1% and then exponential afterward, TCP-Highspeed aims to be fair in high congestion and low RTT connections by acting the same as TCP-Reno. In a long RTT connection, TCP-Reno does not take advantage of the bandwidth that the exponential HS-TCP can. A normal TCP-Reno connection window is found with $w = 1.2/\sqrt{p}$, where p is the packet loss rate and HS-TCP uses the function $w = 0.12/p^{0.035}$. At .0001% packet loss, HS-TCP has a 4.7 Fairness index or is expected to obtain 4.7 bandwidths worth of TCP-Reno Flows, and this increases to a 10.2 Fairness index at .000001% packet loss rate. HS-TCP suffers in any unreliable network and will perform the same as TCP-Reno until the packet loss is less than 0.1%.

Scalable-TCP (Kelly, 2003) is a further development of Highspeed-TCP where the CWND reduction is set to .875 instead of .5. This allows for a recovery time to be consistent for all bandwidths. Its additive increase is designed to double its bandwidth every 70 round trip times for any rate. This more aggressive highspeed TCP has a faster recovery time but may be more unfair to other TCP flows, especially at higher rates. At higher loss rates, 1% to .1% scalable TCP's window size does grow as quickly as TCP-Reno's and so is unfair to itself in higher congestion networks, while unjust to TCP-Reno in lower congestion networks. BIC-TCPs

paper (Xu et al., 2004) claims that scalable-TCP has the worst RTT-unfairness, citing that a short RTT flow will eventually starve a long RTT flow.

H-TCP (Leith and Shorten, 2004) is a long RTT high bandwidth algorithm with an emphasis on fairness and convergence speed. H-TCP relies on the fairness created in the additive increase A in TCP-Reno and its CWND decrease B, with $A = 2(1-B)$ for fairness. H-TCP is a TCP-Reno modification and relies on TCP-Reno to continue to be widely deployed. H-TCP relies on an exponential curve created as a function of time since the last packet drop occurred, similar to HS-TCP. In low RTT networks, packet loss happens frequently, and a mode switch makes H-TCP behave the same as TCP-Reno for a time period after the congestion events. In order to achieve better fairness in long RTT, high-bandwidth bottlenecks, an adaptive backoff algorithm is used to maintain a low CWND reduction.

BIC-TCP, Binary Increase Congestion Control, is the predecessor to CUBIC and was selected as the default TCP congestion control algorithm by Linux in 2005, used in Linux kernels 2.6.8 to 2.6.18. It is more aggressive and less fair to TCP-Reno flows but has good stability, window scaling, and round-trip-time fairness (Xu et al., 2004). This was later modified into TCP-CUBIC, the now default CC for Windows and Linux. BIC-TCP was designed primarily to balance RTT unfairness, scalability, and TCP friendliness without being a perfect solution.

For standard AIMD style congestion control like TCP-Reno, its RTT unfairness can be described by $(RTT_2/RTT_1)^2$. To solve this, larger flows will have to decrease their CWND and shorter flows increase the CWND during convergence despite their RTTs. BIC-TCP has two layers: packet loss-only detection and congestion control. On the first layer, the previous max bandwidth before congestion is set (as W_{max}) and BIC-TCP reaches this level in a concave manner, increasing congestion quickly when far and more slowly when close to the threshold.

BIC-TCP introduces Fast Convergences, where every time the W_{max} is reduced, the W_{min} thresholds are also reduced. Between a reducing flow and a new flow, the shrinking flow will not increase bandwidth as quickly as the new flow increases on the network. This allows the smaller flows to catch up to larger flows. To prevent BIC-TCP from being too aggressive or overshooting on shorter networks, BIC-TCP uses an additive increase if the CWND increase is too great. This means that when recovering from congestion, BIC-TCP does not recover too quickly but does take advantage of reducing the CWND increase close to detected congestion. Once the previous W_{max} threshold has been reached, BIC-TCP begins a convex, exponential, slow-start style probe for bandwidth (Xu et al., 2004).

TCP-CUBIC was first standardized in RFC 8312 and updated in RFC 9438, but had been long described before in 2008 by Ha et al. (2008). It replaced BIC-TCP's binary search with a CUBIC algorithm and updated the slow start to a new hybrid start algorithm (Ha & Rhee, 2011) designed to exit slow start before creating massive packet loss. It replaced BIC-TCP as the default Linux congestion control since Linux kernel 2.6.19. Its decade-plus deployment on the real internet has shown that it is safe and offers a better user experience than the classic Reno congestion control. It is now the most widely deployed congestion control algorithm and has been used in other protocols, like QUIC and SCTP, for congestion control. TCP-Reno's linear increase to the congestion window can be considered slow in high bandwidth, large latency networks. This problem exists for all Reno-style congestion controls, such as NewReno, SCTP, TCP Friendly Rate Control (TFRC), and the default QUIC congestion control (Xu et al., 2023).

CUBIC is designed with a CWND growth to follow a CUBIC curve, so when congestion control is activated, it quickly increases the window size at each ACK and decreases the amount it increases each time, reaching a plateau. This allows it to reach the congestion window quickly

but not supersede it for a time, which is very similar to BIC-TCP. After some time, CUBIC will then attempt to exceed the congestion window in an exponential fashion, testing the network for increased bandwidth. When congestion is found, it again backs off by half, quickly increases close to the congestion window allowance, and sends data at the detected acceptable rate. The RFC describes this as "concave and convex profiles of a CUBIC function." CUBIC is meant to be Reno-friendly and behave like Reno in short RTT networks. TCP CUBIC tries to maintain RTT fairness by increasing its CWND by real-time increase instead of by each RTT and ACK received. This means a high RTT flow and a low RTT flow should attempt to increase their CWND at the same time interval instead of relying on the RTT. The YeAH-TCP, Baiocchi et al. (2007) paper noted that TCP CUBIC is not perfectly fair in short RTT situations and fairer than TCP-Reno in unfair RTT situations.

YeAH-TCP (Baiocchi et al., 2007) which stands for "yet another highspeed," claims that other high-speed congestion controls fail to prevent packet loss events in order to focus on high link utilization. These algorithms do not attempt to avoid congestion. YeAH-TCP goals include efficiency, not stressing the network more than TCP-Reno, TCP friendliness, RTT fairness, and random link loss durability. It uses two modes, fast and slow, to control its CWND. In Fast mode, it acts like Scalable TCP for simplicity, and in the slow mode, it acts as TCP-Reno. It will decide on its mode by using a bandwidth delay product estimate similar to TCP-Westwood. If no queue increase is detected, then it acts in fast mode. Scalable-TCP's 12.5% CWND decrease in this mode is helpful for radio link-loss durability. It also maintains a counter for fast mode and competing Reno flows, so it can determine if it is competing with greedy TCP-Reno flows and switch to its Slow mode to not out-compete and also fill the bottleneck buffer. When there are no competing greedy flows, it keeps the bottleneck queue low. Using the BDP (bandwidth delay

product) estimate, when it receives 3 ACKS, it can determine the number of packets it needs to decrease in order to empty the bottleneck buffer while keeping the pipe full. It has good convergence with itself, maintains good efficiency in higher RTTs compared to other high-speed TCPs, and has better radio link-loss tolerance over other high-speed TCPs, at 0.00001 packet loss and better. It was found to be very RTT fair, with H-TCP in second and CUBIC in third.

TCP-Illinois (Liu et al., 2006) relies on creating a concave response curve CWND instead of the convex curves used in HS-TCP, H-TCP, and BIC-TCP. It uses packet loss information as a primary congestion signal and queueing delay as a secondary congestion signal. The context curves in H-TCP and HS-TCP mean that there is a lot of available bandwidth that is not explored and quickly backed off from when overshoot, also creating many congestion events and being unfair to larger RTT flows. By using queueing delay, when the delay is small, TCP-Illinois sets a large additive A increase and a small CWIND decrease B, which can also help in radio link loss packet loss scenarios. When the queue delay increases, A decreases and B increases to that of regular TCP-NewReno. All features of TCP-NewReno, except the AIMD algorithm, are retained. SACK is used but is less necessary as this algorithm creates fewer congestion events. If claims are accurate, then TCP-Illinois is better than CUBIC-TCP, which is evaluated in this experiment.

Random and Link Loss TCP Congestion Control

The original TCP congestion controls assumed a wired network, where 99% of packet loss occurs due to congestion. The popularity of wireless networks has motivated the creation of congestion controls that can differentiate packet loss due to radio link loss from congestion.

TCP-Westwood, 2001, uses a bandwidth estimator by measuring the interval of returns ACKS along with a low-pass filter (Mascolo et al., 2001). It is extremely effective in hybrid networks, as it can differentiate random loss from radio interference and congestion packet loss

(Kharat Prashant, 2019). TCP-Westwood+ was an update in 2004 to correct an issue with compressed ACKs not allowing the bandwidth estimate to work correctly. When congestion is detected, Westwood+ sets the SStresh to the bandwidth estimate / RTT and then proceeds as TCP-Reno (Dell'Aera et al., 2004). The bandwidth estimate is calculated for every other RTT. This method, while helpful in ignoring random packet loss, may not be fair to other flows and also, if the bandwidth estimate / RTT is accurate, does not gain any additional bandwidth after the SStresh is reset and only increases RTT on each ACK. Also, this method does not take advantage of large increases in available bandwidth that may occur in high RTT high bandwidth available connections. It claims to have 23-53% better goodput than TCP-Reno.

TCP-Veno, 2003, combines TCP-Reno congestion control with TCP-Vegas actual vs expected RTT estimate to determine if packet loss is caused by radio link loss or congestion. TCP-Vegas by itself is at a disadvantage when competing against TCP-Reno flows as Reno is more aggressive and eats bandwidth until packet loss occurs, while Vegas is proactive and attempts to back off before a congestion event, despite the original TCP-Vegas paper claiming an estimated 50% better throughput. TCP-Veno competes with Reno by being very similar. Suppose congestion is detected with the Vegas RTT estimates. In that case, Veno only increases the CWND every other RTT, meaning it will stay closer to the optimal bandwidth for longer while probing for congestion and creating fewer congestion events. If packet loss does occur, but the Vegas RTT is not detecting congestion, then that loss is assumed to be radio link loss, and the CWND is only reduced by 1/5th (Fu & Liew, 2003). The TCP-Veno paper showed that Veno should have around a 150 Kbyte throughput in a 1.6Mbps, 120ms RTT network.

TCP-CDG, or TCP CAIA Delay Gradient, 2011, is another attempt to use RTT to detect packet loss in radio link loss vs. congestion. It claims an 84% better bandwidth over NewReno in

a 1% link loss error rate network. Instead of using a single RTT measurement for base RTT, it uses a moving average RTT estimate to detect congestion vs. radio link loss, as RTT is a noisy signal. Larger RTTs will have larger variances than small RTTs. The CWND reduces to 70% every 2 RTTs when congestion is detected. If reducing the CWIND does not reduce the RTT, TCP-CDG will exit congestion avoidance and attempt to probe for more bandwidth again. TCP-CDG does not claim to be better than NewReno when there is no radio link loss error rate (Hayes & Armitage, 2011). In a 1% packet loss environment, TCP-CDG claims to achieve 65% of available capacity compared to 35% for NewReno.

TCP-BBR (Cardwell et al., 2017) was made by Google. It tries to probe for the BDP, bandwidth-delay product, by having a startup similar to a slow startup until it notices an increase in RTT, then "drains" the window until it meets the highest bandwidth with the lowest RTT. After that, it will periodically probe for more bandwidth, then lower RTT sequentially and adjust accordingly. The bottleneck bandwidth and the practical RTT will also vary independently during the connection and must be constantly monitored. This will theoretically send data at the most efficient data rate and account for a lossy link. BBR does not reduce its CWND on packet loss but on RTT increase, significantly helping radio link loss packet loss recovery as all packet loss can be treated as error and all RTT increase is treated as congestion. Old congestion controls also relied on smaller buffer sizes that were quickly filled. However, in larger networks with more bandwidth and larger buffers, the buffers may take some time to fill before packet loss occurs, only increasing RTT on the flow without ever improving throughput. Google had set the original BBR to be functional with up to 15% loss compared to CUBIC, which can lose over 50% of the available bandwidth with only a .01% error rate. These claims are tested in this experiment. It has

been deployed in Google and YouTube servers with dramatic results, with the most in developing areas.

Google designed BBRv2 to have better fairness to legacy TCP flows than BBRv1. It also has ECN support and uses packet loss as a congestion indicator (Drucker et al., 2024). BBRv2 has a 2% loss threshold that significantly reduces its sending rate, whereas BBRv1 could operate with up to 15% losses. It greatly reduces its throughput compared to BBRv1 in the interest of better fairness to TCP-CUBIC and TCP-Reno. BBRv3 was created in 2023 to address more issues that occurred in BBRv2, including convergence issues with other BBRv2 flows (Gomez et al., 2024). BBRv3 has similar fairness to TCP-CUBIC as BBRv2 but has improved the throughput to be closer to BBRv1. Neither BBRv3 nor BBRv2 are available in the most recent Linux Kernel (6.13.7 on March 18th, 2025), but BBRv2 is available in the Windows 10 kernel.

Satellite Network Designed Congestion Control

TCP-Hybla, 2004, works by modifying the slow start to have a fixed RTT goal so that the slow start ramps up regardless of RTT. Additional improvements in congestion control neutralized the detrimental effect of RTT. This is done by estimating the additional RTT time the radio link adds to the satellite and reducing the time needed for TCP reactions by that amount. This is combined with the SACK option to retransmit missing packets when loss occurs. The goal of TCP-Hybla was to fix the performance detriment caused by high RTT in satellite networks, and it does not effectively discriminate between congestion packet loss and link error packet loss. Connections with high RTT tend to "starve" in the presence of other TCP flows without high RTT, which can adapt to congestion signals more quickly and take up bandwidth available more quickly (Caini & Firrincieli, 2004). TCP-Hybla does not need modifications of the router between sender and receiver and only an optional modification for the receiver. TCP-

Hybla+SACK had greatly improved performance over TCP-Reno, NewReno, and Vegas, specifically the worse the RTT time was. Correcting for link-loss and congestion is less of an issue than the sum of the issues because the correction for link-loss, though degrading possible performance, reduces the likelihood of encountering congestion packet loss.

Other TCP Congestion Controls

DCTCP (Data Center TCP) is a solution to some issues that occur specific to a data center or data retrieval environment. In a case study example, a 6000-server cluster had high application latency due to high buffers from TCP demands. This occurred when TCP requests went to multiple servers that tried to reply at the same time, filling the switch buffers and greatly increasing latency on the services requested. Even though there was ample bandwidth, the initial spike of TCP flows would clog at least one flow and drop packets, leading to high latencies. DCTCP uses explicit congestion notification and adds intentional jitter to TCP requests, leading to requests being intentionally delayed by a varying few milliseconds. By delaying requests by varying milliseconds, the initial burst of data gets smoothed out over a few milliseconds, slightly decreasing the average response time by a few but greatly reducing the outlier response time. For example, when loading a webpage with images from many servers, all the images would normally try to load at once, filling the switch's buffers and causing packet loss. By staggering the TCP requests by a few milliseconds, the buffer will not be filled by all the requests at once, and all the requests can be delivered without packet loss. The explicit congestion notification (ECN) is a router feature where a longer TCP flow can get a notification that the buffer is filling, and it can enable congestion control and back off its traffic before itself or any other TCP flows experience packet loss (Alizadeh et al., 2010). Due to the use of ECN and the artificial, varying

delay of TCP transmissions, DCTCP is not well designed to be used outside of a data center. DCTCP was not tested in this laboratory.

TCP-LP, Low Priority (Kuzmanovic and Knightly, 2003), is a TCP congestion control variant designed not to compete with traditional TCP flows but to try and grab excess bandwidth available on the network without interrupting other flows and being fair to other flows of itself. It does this by using the TCP timestamp feature to detect growing RTT time and backing off even before detecting growing RTT. In practice, TCP-LP found significant bandwidth available while not interfering with regular TCP-Reno flows. TCP-LP was tested with other congestion algorithms in this experiment, such as CUBIC, Vegas, NV, and BBR.

MPTCP

Multipath TCP is an extension of TCP that allows an application to use TCP over multiple connections if, and when, it is available. It is currently described as version 1 in RFC 8684 (Ford et al., 2020), which replaced version 0 in RFC 6824 (Ford et al., 2013); the two versions are not cross-compatible. The multiple connections allow higher bandwidth and reliability. The current implementation in Linux Kernel is of MPTCPv1 (MPTCP Dev Team, 2024) and Linux has had MPTCPv1 support since kernel version 5.6. The current tracked changes can be found at (Linux MPTCP Upstream Project, 2024).

MPTCP begins with a regular TCP connection, which includes an MP-CAPABLE flag on the three-way handshake. If the host and server are both capable, some authentication information is included in the handshake to open new subflows. A packet scheduler sits above the multiple subflows and congestion control algorithms and decides on which path the next packets of data are sent. MPTCP supports changing potential IP addresses being added or removed from the connection. Using a data sequence number on top of each subflow's own

sequence number allows MPTCP to retransmit data on different subflows in the event of failure. Some notable features include coping with NAT on path by referring to addresses with address IDs, being able to fall back to single-path TCP, and sending security keys to avoid malicious use of the MPTCP subflows by attackers (Ford et al., 2020).

MPTCP can help hand-offs between connections; if a TCP connection has to terminate and establish a new connection when switching networks in the middle of an application stream, MPTCP can hand off the connection between two networks without interruption. Ultimately, MPTCP can offer 3 improvements of TCP: higher throughput with bandwidth aggregation, higher resilience, and better network resource utilization. (Jowkarishasaltaneh and But, 2022). The standard packet schedulers in MPTCP can have issues as RTT and bandwidth between paths change, as differing orders of packet arrival will cause head-of-line blocking, where data cannot be processed until an earlier sequence-numbered packet arrives. Congestion controls and schedulers with reliable and predictable RTTs are preferred to avoid head-of-line blocking. Optimizing for bandwidth can support congestion control and schedulers that tune for throughput.

Coupled Congestion Control

MPTCP will use at least two subflows, which should act fairly on the network, and congestion control is either coupled or uncoupled. In uncoupled congestion control, each subflow will have its own independent congestion control and congestion control window (CWND). With coupled congestion control, the congestion window is shared across all subflows. Coupled congestion control often behaves with the assumption that there will be a bottleneck between two or more subflows and their receiver. The coupled congestion control algorithms are unique for multipath protocols like MPTCP and MPQUIC, and can include the following types: Linked

Increased Algorithm (LIA), Opportunistic Linked Increase Algorithm (OLIA), Balanced Linked Adaption Congestion Control Algorithm (BALIA), and Weighted Vegas (wVegas) (Jowkarishasaltaneh and But, 2022). Coupling congestion control in MPTCP helps ensure fairness with other TCP flows on the network. Otherwise, a single MPTCP connection can take the bandwidth of two or more TCP connections, which could be considered unfair. In order to decide what channel to send and what order of packets of data to send, a scheduler must sit on top of the TCP subflows and decide at what time and which subflow to send a packet of data. A good scheduler is needed in order to prevent head-of-line blocking. The coupled congestion control can be paired with the uncoupled congestion control on a single subflow, as shown in "Evaluating MPTCP Congestion Control Algorithms: Implications for Streaming in Open Internet" (Łuczak et al., 2023).

A good scheduler must integrate with the transport layer and make decisions based on congestion on both paths and RTT on each path. With equal bandwidth, to prevent head of line blocking, a path with a 100ms RTT will only be able to send 1 packet per each 2 packets on a 50ms RTT path. Many connections will not be this extreme and likely have closer to a 60ms/50ms difference between paths, for example, which will still cause a 10ms head of line block if data is sent incorrectly. The scheduler can also be optimized for its purpose, and machine learning can even be integrated in order to make the best decisions for its purpose, such as with the Peekaboo scheduler or ReLeS scheduler (Vu et al., 2022).

A packet loss-based congestion control will fill buffers of routers in between the endpoints, and MinRTT, the default scheduler, will prefer the subflow with lower RTT until it starts to fill the buffer of the router to the point its RTT will overcome the other optional subflow, in which case it will switch to that flow. This likely works very well in a wired environment with

similar RTT on paths and using CUBIC or TCP-Reno that will send data to fill buffers before reducing their congestion window. This could also mean that one path will maintain a buffer on a router while only maximizing the other path's RTT. If the RTT of one path is too high compared to the other, it can never be used efficiently, which could be an issue for dual connections with a satellite connection.

As of Linux Kernel 6.6.59 long-term kernel, MPTCP does not appear to have a coupled congestion control option. The four popular coupled congestion controls, LIA, OLIA, BALIA, and wVegas, are absent (Linux Kernel Development, 2024). This either means LIA is absent or is always enabled by default, and single path congestion controls are options underneath.

LIA stands for "linked increase adaption" and should be the standard CC for MPTCP flows according to RFC standards (Ford et al., 2020). This congestion control assumes AIMD, additive increase, and multiplicative decrease and can have detrimental effects on CUBIC TCP and other multiplicative increase congestion controls. It is found in RFC 6356. The original goal of LIA CC is to treat MPTCP as a single connection with more capacity. This means a double 100mb/s connection with MPTCP should be treated as a single 200mb/s connection in congestion control. This is achieved by linking the subflows' CWND increase functions. The goals are to increase throughput to be at least as good as the best subflow alone, fairness to other connections, and balance congestion away from the most congested path. Detecting shared bottlenecks is noted to be difficult. LIA only affects the increase rate for CC, whereas slow start, fast recovery, and fast retransmit are the same as standard TCP and stay on subflow only. Each Acknowledgement allows a subflow to increase its congestion window by its current CWND divided by the total CWND. So, when the total CWND is high, the increase in the individual CWND is low. If the individual CWND equals the total CWND, then the multipath congestion

control will behave the same as a single TCP flow. For example, if a single connection has many congestion incidents and its multipath partner has few, then its individual CWND will be much less than the total CWND, allowing a better congestion path to be preferred for CWND growth. A variable α is calculated to adjust the aggressiveness of the multipath flow (Raiciu et al., 2011).

OLIA was built to fix an issue with LIA, which can reduce other TCP flows while also not necessarily benefitting the MPTCP flow. OLIA uses the CWND in the current subflow instead of the maximum CWDN as the basis for its calculated CWND increase (Kato et al., 2019). OLIA was shown to fix LIA pareto-optimal issues and was also better than LIA in every case in a wireless network (Khalili et al., 2013). OLIA is also the default congestion control for MPQUIC (Viernickel et al., 2018).

BALIA focuses on balancing the congestion windows of different paths. By doing so, BALIA ensures that no single path is overly congested, leading to better overall network performance. BALIA is particularly effective in scenarios where the available paths have diverse and variable characteristics (Łuczak et al., 2023).

wVegas is a delay-based congestion protocol that adjusts the CWND on each path to minimize delay. The concept is the same as TCP-Vegas, which is that routers mid-path will buffer traffic as congestion increases, signaling an increase in congestion. By reducing the CWND in response to increases in RTT, congestion events can be avoided. In actual implementation, this may not be as effective as planned (Łuczak et al., 2023). Other congestion protocols that are not RTT aware may increase the router's buffer as they attempt to probe the network for bandwidth, and as such, Vegas and wVegas will back off their CWND in response to other flow's bandwidth probing.

"Experimental analysis of MPTCP congestion control algorithms; LIA, OLIA, and BALIA" (Kato et al., 2019) acts as if MPTCP CC works without a sub-flow CC at all or that both use TCP-Reno as their congestion control. Łuczak et al. (2023) treats MPTCP CC as each subflow has a congestion control, and the entire MPTCP has a coupled congestion control. Above the CC, the packet scheduler can also be optimized. Luczak's study shows the best outcome for CC combinations.

An additional study of a scheduler needs to be attempted to find the best performance for a multipath connection, TCP or QUIC, in different network scenarios. For example, a Wi-Fi and a 5G connection might have slightly different RTT, bandwidths, and link loss error rates, which will require scheduler adjustments and the selection of the best CCs for their flows, depending on the use of the device. A streaming service may be considered the best end-user experience, while the best bandwidth optimizations may be best for data centers or business applications, and finally, the lowest latency and high reliability may be the best for an industrial setting. Another best option for a dual home connection may be a satellite connection, which also has unique congestion control requirements. A geostationary satellite has a consistent high-RTT connection, with some possible radio link loss.

It is interesting that Luczak, in "Evaluating MPTCP Congestion Control Algorithms: Implications for Streaming in Open Internet," studied pairing MPTCP with SPTCP (single-path transmission control protocol) congestion controls, meaning LIA or OLIA could be paired with single-path Reno or BBR. This offered 12 pairs of MPTCP congestion controls with SPTCP congestion controls. Different combinations worked best for preferred scenarios. Luczak found good combinations with TCP Reno, which is interesting as TCP-Reno is older and much less aggressive than BBR or CUBIC.

First, considering just CUBIC, the default TCP CC currently on Windows and Linux, paired with BALIA, offered the best throughput but also the highest drop rate. This means high video quality, but stream breaks and pauses must also be accounted for in buffering. Higher throughputs were found with BALIA with Reno and OLIA with BBR.

LIA paired with BBR and BALIA with Reno performed best to optimize for high throughput and low drop rate. This is opposed to BALIA with CUBIC, resulting in the highest drop rate. BALIA, with BBR, achieved the lowest HoL (Head of Line) blocking. Less head-of-the-line blocking is preferred for streaming high throughput, such as 4K video. For the lowest path delay, wVegas with Reno and OLIA with BBR performed best with the lowest latency and jitter. BALIA with CUBIC and BALIA with BBR performed the worst. This would simulate a real-time application, such as a video conference. So, BALIA with Reno was best for high-resolution streaming, while LIA with BBR was best for packet loss. Head of line blocking was lowest with BALIA with BBR, which is optimal for more resource-constrained devices, such as mobile devices (Łuczak et al., 2023).

Łuczak's study shows that different combinations of coupled and uncoupled congestion controls have different best applications and sometimes surprising results. It is suspected that Reno performed so well against CUBIC and BBR when theoretically limited in probing bandwidth. Łuczak's experiment only tested streams for 10 seconds at a time, not nearly giving the congestion control protocols a chance to differentiate in their ability to probe for bandwidth. So, a more accurate representation of the best MPTCP and SPTCP combinations should exclude TCP-Reno or at least have a longer test period.

While BALIA with CUBIC will hold up with high throughput and high drop rate, BALIA with Reno is unlikely to actually be the best high throughput choice for streaming. BALIA with

BBR is likely the best combination for MPTCP, with the ability to probe for bandwidth and have decreased Head of Line blocking, which becomes more important the more bandwidth is used. It was found to have the worst jitter, so OLIA with BBR would be best for real-time application.

Meanwhile, the actual Linux Kernel implementation of MPTCP does not have options to change the coupled congestion control (Linux Kernel Development, 2024). This experiment evaluates whether MPTCP uses coupled congestion along with single-path congestion control or only single-path congestion control on each of its flows. If MPTCP uses coupled congestion on top of regular congestion control, the only option available is LIA (MPTCP Dev Team, 2024).

Schedulers

In "An Empirical Study of MPQUIC Schedulers in Mobile Wireless Networks," Vu et al. discovered that the best schedulers for a mobile environment are learning-based, with q-ReLeS best for MPQUIC, which is the MPTCP scheduler ReLeS made for MPQUIC (Vu et al., 2022). Vu et al. did not evaluate congestion control along with the schedulers, meaning there is still ample room for research on optimizing multipath connection protocols depending on the purpose of the connections. The default scheduler for MPTCP and MPQUIC is minRTT, which fills the congestion window with the lowest RTT first before sending traffic on other subflows. This does not work well in heterogeneous paths, as one path has different RTT and loss amounts due to being wireless. BLEST, blocking estimation-based scheduler, and ECF, earliest completion first, introduce a waiting mechanism and rely on heuristics. Their concept is that waiting for the fastest path to open up again can sometimes be faster than trying to send data on an open but slower path.

Learning-based schedulers are being researched. Peekaboo uses an initial deterministic decision on which paths to send traffic and adjusts on top of these. When paths are similar,

Peekaboo acts most similarly to the minRTT scheduler. ReLeS uses Deep Reinforcement Learning to solve scheduling dynamics by looking at the past and present state of the network in order to make decisions. ReLeS was made for MPTCP, and the MPQUIC version that was studied is Q-ReLes (Vu et al., 2022). The study used a mininet-Wi-Fi to simulate a cellphone connected to a wireless access point and a 4G LTE tower, and to walk around in different ways.

The congestion controls were not mentioned, and so can be assumed to be the default, OLIA for MPQUIC and LIA if for MPTCP. Even as a controlled variable, though, congestion controls such as CUBIC on each subpath could have one subflow stuck in a congestion avoidance phase while another begins to probe for more bandwidth. This still may be best used along with a deep learning scheduler, as that may be best able to handle the dynamics of a changing network environment with congestion controls probing network bandwidth.

The study showed that the worst scheduler was the default min RTT, but this was still a better performance than single-path TCP or QUIC alone. ECF and BLEST schedulers were not shown to perform much better than minRTT. Peekaboo showed better performance than the non-learning-based schedulers, and Q-ReLeS showed the best performance, doing better than Peekaboo by 10-20% (Vu et al., 2022).

QUIC

QUIC was first designed in 2012 by Google to be a reliable transport protocol over UDP, with a built-in TLS connection in the first handshake (Langley et al., 2017). This solved two issues, the first being the ossification of TCP in middleboxes on the internet, and the second, a faster connection time before the client can send application data by reducing the number of RTTs needed to establish a secure connection. Utilizing UDP allowed QUIC to be deployed without changing any of the infrastructure hardware on the internet. IETF eventually

standardized QUIC as RFC 9000 (Iyengar and Thomson, 2021) and is an integrated part of HTTP/3 (Bishop, 2022). By being developed by Google, it has been widely deployed on YouTube and by competing technology companies such as Meta and Amazon.

QUIC provides a faster handshake by incorporating the TLS connection in the initial handshake, so instead of two or three RTTs to establish a secure connection, it can take one or even none. This entire one less RTT means that worse RTT connections have a better performance boost when switching to QUIC. Zero RTT is accomplished by having a session saved with the server and immediately sending application data with the saved session keys. Multiplexing occurs in QUIC, similar to HTTP/2, where multiple independent stream numbers can be used, and if one UDP stream loses a packet, only that stream needs to be corrected. These stream ID numbers also persist if a user changes IP address, smoothing data migration and benefiting mobile users. Additionally, packet transmission has its own packet tracking ID so that when packets are re-transmitted on the network, the original packet and the re-transmitted packet can be differentiated if both arrive, as both have the same sequence number but different tracking IDs (Iyengar & Thomson, 2021).

The difference between TCP and UDP originally was reliable transmission. TCP eventually developed congestion control algorithms to help pace data transmission on a network. QUIC handles the congestion control at the application layer rather than the transport, meaning that QUIC may not be able to adopt any of the routing features that TCP has implemented on middleboxes. Effective congestion control is a priority on highly efficient networks. Fairness is essential as higher level flow control and quality of service can be implemented to prioritize application types in network management. QUIC protocol can obfuscate the application type, which can be a network management challenge (Kharat Prashant, 2019). Both QUIC and TCP

share many of the same features, such as congestion control, which is why the proper configurations are crucial for connection performance, even between HTTP/2 and HTTP/3. For example, a TCP connection with a radio link loss-aware congestion control is going to perform better than a QUIC connection with a classic congestion control on a wireless network, and this simple configuration difference can make an HTTP/2 connection with TCP more efficient than an HTTP/3 connection with QUIC.

In 2017, QUIC was only 7% of internet traffic (Langley et al., 2017), and according to Enotiadis' NANOG conference talk, QUIC was 50% of network traffic in 2023 (Enotiadis, 2023). According to Cloudflare Radar, in June of 2024, QUIC and HTTP/3 are 30% of the internet, while 96.5% is HTTPS (Cloudflare, 2024). Websites like Facebook now have nearly 80% QUIC traffic. HTTP/3 with QUIC, DNS over encryption, and eSNI/ECH, all encrypted over TLS 1.3, is the new network stack that network managers must handle. This network stack significantly obfuscates some of the data that network managers used to rely on for network management. Enotiadis recommends time-domain shaping to correct the issue; network data in TCP that used to be used for traffic shaping is now hidden in the QUIC protocol. As QUIC is more efficient and aggressive at recovering from packet loss, it can often gain an unfair size of the network bandwidth compared to TCP (Enotiadis, 2023). This means that for the end user, applications have a better QoE (Quality of Experience) while possibly being unfair to other connections.

MPQUIC is very similar to MPTCP but has the additional benefit of not needing to be deployed at the operating system kernel. For MPTCP, an update to the Linux Kernel or Windows Kernel needs to occur for the protocol to function. With MPQUIC, as it is deployed at the application layer, the operating system kernel does not need to be modified for it to be deployed

(Viernickel et al., 2018). This may be a big reason why MPTCP has not been commonly deployed, as doing so requires an operating system update, and applications must be configured to attempt to run MPTCP.

MPQUIC

MPQUIC is to QUIC as MPTCP is to TCP. It is still in internet draft (Viernickel et al., 2018). Modules have been developed to test MPQUIC in the NS-3 simulator (Shu et al., 2023). An Advantage of MPQUIC over MPTCP includes new MP_ACK frames as opposed to the original ACK frame. ACK frames in a multipath system must be sent back the same path as the data received, but MP_ACK frames can be transmitted on any path available (Liu et al., 2024). This may have improved performance, but it may also create new security considerations.

Reviewing congestion control algorithms suggests multipath protocols would be even more affected by radio-link loss, causing unnecessary throughput reduction and increasing out-of-order packet arrival, creating head-of-line blocking. A radio link-loss congestion control protocol, such as BBR from Google, would be useful to remedy this. Having each individual QUIC flow at a stable output of BDP will help estimate the correct sending order, which is achieved in BBR's bandwidth-delay product estimation. Using traditional single-path congestion protocols leads to unfairness by the multipath QUIC stream toward the single-path streams. The original LIA coupled congestion control noted that bottleneck detection would be very useful but difficult to implement; it would indicate the congestion control when and where multiple streams are starting to compete with other streams and then reduce the congestion window fairly in these instances (Raiciu et al., 2011).

Ideally, with no bottleneck, each path can maintain a single path style congestion control. Combining BBR with MPQUIC has been proposed by (Deng et al., 2024), (Mahmud et al.,

2020), and (Wei et al., 2020). (Wei et al., 2020) and (Deng et al., 2024) included a bottleneck detection system, only coupling the congestion control window reduction when a shared bottleneck is detected. If sufficient, efficient, and fair, this could be a solution to better network resource utilization with multipath protocols.

In Zheng et al. (2021), researchers devised a receiving device that sends QoE data back to the sending device in MPQUIC that informs the scheduler of the order that packets are being received, at which the scheduler can adapt to optimize the subflows for quality experience based on optimization for short video playback. Their large-scale deployment test of 3 million e-commerce product short videos showed a 19-50% improvement in video-chunk request completion time, 32% improvement in first-frame latency, and 23-67% improvement in re-buffering at the expense of only 2.1% redundant QUIC traffic. Their deployment also demonstrates the flexibility of an MPQUIC deployment and configuration adjustment not limited to a kernel update as with MPTCP.

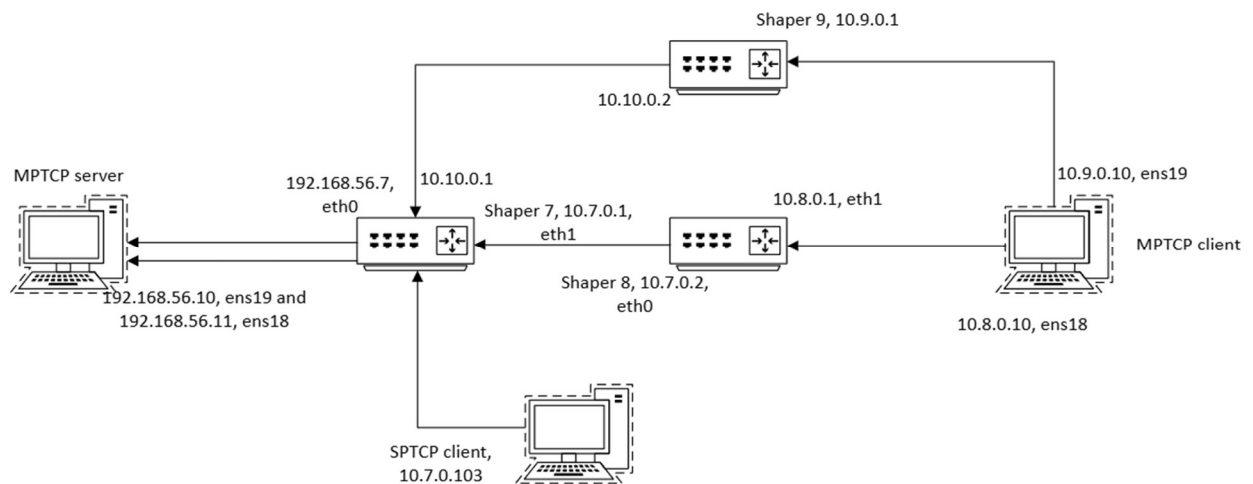
Chapter Three: Methodology

Design

A Proxmox hypervisor is installed on an AMD FX-6300 computer with 8GB of DDR3 RAM and a two terabyte HDD. On this, three virtual machine endpoints with Ubuntu 22 or 24 are installed with "iperf3" and "mptcpize" installed. Three Debian 10 containers are configured as routers and traffic shapers for simulating the network with "tc netem" (Traffic Control, Network Emulation) command. The far-right Ubuntu 24 acts as a sender and is connected to two containers, simulating a network connection to the far-left Ubuntu 22 machine. In the middle of the paths is a second Ubuntu 24 machine and another traffic shaper. One path from right to left simulates a network with another machine sending traffic at a lower RTT. The other path simulates the second MPTCP connection. Figure 1 presents the network diagram, with the Debian routers imaged as network routers, and the Ubuntu virtual machines imaged as desktops. The arrows show the network connections next to the IP addresses, ports, and subnets used in this laboratory. The "tc" command is used on the Debian routers to act as traffic shapers. The

Figure 1

Network Diagram of Proxmox Virtual Laboratory.



laboratory uses the "netem" option to add delay, random packet loss, and set rate limits on traffic leaving the shapers. The "tc" command also has an ECN function, which won't be used here.

The tests were run in November 2024, the VMs were shut down and started again, configurations were checked, and tests were rerun in January 2025 to verify results.

Jain's Fairness Index (Jain et al., 1984) is employed in the second test to assess the relative fairness of MPTCP when competing against a single-path TCP connection. It is a widely used metric in network research, with a 0 to 1 scale, where 0 represents perfect unfairness and 1 represents perfect fairness. It is unaffected by the number of flows, allowing for comparisons between large and small network fairness, and is computationally simple. It can smooth out resource allocations to show entire network fairness. Below is the Jain's Fairness index equation, simplified for this study:

$$\frac{(Flow1^2 + Flow2^2 + \dots + FlowN^2)}{N(Flow1 + Flow2 + \dots + FlowN)^2} = Fairness\ Index$$

The Following Tests Were Performed:

1. Test if MPTCP is affected by setting uncoupled congestion control on the host Linux machine, using a simulated random packet loss on the path and comparing default congestion control to a BBR congestion control, which will handle the random packet loss well.
2. Test if MPTCP is unfair to other flows by relative resource allocation and by Jain's Fairness index. Two MPTCP CUBIC subflows will compete with one SPTCP CUBIC flow on a bottleneck.
3. Test if TCP-LP finds bandwidth when competing with TCP-Reno without bothering the TCP-Reno flow.

4. Evaluated TCP-CUBIC, BBR, Illinois, and YeAH in increasing random link loss percentages.
5. Test wireless link loss congestion controls competing with a default CUBIC congestion control on a simulated wireless link loss bottleneck.
6. The best congestion control to use with MPTCP when sending data to a simulated smartphone with 5G and Wi-Fi 6 connection is to run the tests once in an empty network and once in a congested network.

Chapter Four: Results

Is MPTCP Using Uncoupled Congestion Control?

The first test is if MPTCP is uncoupled and uses single-path congestion control. This is found to be true. This test uses Shaper 8 and 9 with an 800Mb/s bitrate and Shaper 7, the bottleneck, to have a .1% error rate and a 40ms delay. Using a single path connection with CUBIC, we found that the connection begins quickly at 535Mb/s and quickly falls until it reaches an 11.5Mb/s connection after 10 seconds. Running a single path connection with BBR quickly finds 450Mb/s and maintains this connection over its life. An MPTCP connection with CUBIC quickly reaches 740 Mb/s but quickly falls to between 44 and 60 Mb/s, which is still more than four times the speed of the single-path connection. A BBR connection with MPTCP finds 850Mb/s after a minute-long connection, nearly twice the speed of the single-path connection. This implies that MPTCP is significantly affected by the congestion control choice, uses single-path congestion control, and has excellent performance gains even when not using optimized TCP congestion control selections. Either MPTCP uses single-path congestion controls on top of LIA coupled congestion control, or LIA is not implemented on MPTCP in the Linux 6.8 kernel.

Is MPTCP Fair?

Second, the laboratory tested if MPTCP was fair, and it was found to be consistently gaining more bandwidth than its competing single-path flow, with 12.5-40% more bandwidth, but it was found to be highly fair using Jain's Fairness Index.

Using Shaper 8 and 9, we have a set rate of 800 Mb/s each, and on Shaper 7, we have a bottleneck of 1.6 Gb/s. Singlepath TCP with CUBIC finds from SPTCP-client 1.53 Gb/s speed,

as does the MPTCP from MPTCP-client, which finds 1.51 Gb/s when its traffic is split into two 800Mb/s flows. However, when both these flows are run simultaneously for 90 seconds, the single path TCP flow finds 683 Mb/s and MPTCP finds 842 Mb/s speed. 842Mb/s divided by 683Mb/s shows the MPTCP flow finds 1.233 flows worth of relative bandwidth allocation, indicating a slight advantage in resource allocation. Using Jain's fairness index (Jain et al. 1984), we get a .989 fairness index, which is considered highly fair. Below is the equation and its result:

$$J = \frac{(842 + 683)^2}{2(842^2 + 683^2)} = \frac{2,325,625}{2,350,906} = .989$$

Jain's fairness index runs from 0 to 1, where 1 = perfect fairness. A score of .989 indicates that while MPTCP is not perfectly fair, it is relatively close to perfect fairness. In the context of computer networks, this can be an acceptable trade-off in exchange for more efficient network utilization. In an environment where there are many more flows, being 123% more efficient than any one individual flow among tens, hundreds, or thousands of flows may not be considered that unfair.

Another test, run with a 1Gb/s rate limit on Shaper 8 and 9, a 5ms delay on all shapers, and a 2Gb/s rate limit on Shaper 7, found the single path TCP with 813 Mb/s and MPTCP with 1.09Gb/s, which comes to 1.341 flows worth of bandwidth. Jain's fairness index was calculated as .979.

Next, test this configuration again with more bandwidth. With the bottleneck at 4Gb/s and Shapers 8 and 9 at 2Gb/s, we find that SPTCP has 2.16Gb/s bandwidth and MPTCP found 1.65Gb/s bandwidth, .764 relative flows worth of bandwidth. This was an anomaly, as a later test with the same configuration found that SPTCP has 1.58Gb/s with these settings and MPTCP has 2.21Gb/s, which concludes with 1.399 flows worth of bandwidth. This indicates an inconsistency

with MPTCP finding bandwidth in a congested environment. Jain's Fairness Index was calculated to be 0.973.

When two Single-path TCP connections were used, both quickly found 1.9Gb/s bandwidth. With the bottleneck set to 400MBytes/s, SPTCP found 1.44Gb/s consistently, and MPTCP found 1.62Gb/s consistently, except occasionally, SPTCP could achieve 2Gb/s and MPTCP would be stuck at 1Gb/s for the majority of its flow. The conclusion is that MPTCP is inconsistent at finding fairness, often achieving 1.125-1.4 when having two flows. This could mean that MPTCP, in its current form, finds roughly $(N-1)*1.4$ worth of flow's bandwidth allocation (for at least CUBIC congestion control). Further research could be conducted using more network interfaces and paths to see if more network connections would indeed allow MPTCP to act greedier. Nonetheless, for a dual-homed smartphone connection using MPTCP, Jain's fairness index indicates that the flows were close to fair, with the worst case having a .973 fairness index.

TCP LP for Bandwidth Without Interrupting Other Flows

The next test is for TCP-LP (Low Priority), which can send data over a network without interrupting another TCP-Reno flow. Shaper 7, the bottleneck, is set to a 20ms delay and a 1Gb/s bandwidth. As TCP-LP is meant to use timestamps, the delay is set to 10ms going left and 10ms going right on the network. Tests found that at 1Gb/s, TCP-LP took an even share of the network along with the competing TCP-Reno flow, acting as a normal TCP-Reno flow instead of not interrupting traffic. For example, coexisting with another 20-second TCP-Reno flow, TCP Reno found 488 Mb/s and TCP-LP found 485 Mb/s. The TCP-Reno flow, by itself, found 944 Mb/s. The TCP-LP flow, by itself, varied wildly from 938Mb/s to 71.3 Mb/s, and found an average of

584 Mb/s. Later tests showed TCP-LP acted stable and found a bandwidth of 955Mb/s. TCP-LP was found to be broken in implementation in Linux Kernel 6.8.

Instead of using TCP-LP, TCP-Vegas does a good job of having a low bandwidth and minimal interruption to other flows. TCP-Vegas achieved 58.2Mb/s while TCP-Reno had a 927Mb/s bandwidth. TCP-Reno only lost 17Mb/s while TCP-Vegas found 58.2Mb/s available bandwidth. A test with TCP-CUBIC found 931Mb/s with TCP-Vegas finding 26.1Mb/s.

Testing Bitrate With Random Packet Loss With CUBIC, BBR, Illinois, And YeAH

TCP CUBIC is not radio link loss tolerant and is the default congestion control algorithm for many operating systems. BBR is claimed to have good radio link loss tolerance up to 15% error rate. In the test, we gradually increase the error rate to see the tolerance of each congestion control. Delay is set to 5ms and rate control is set to 3Gbit/s, on Shaper 7 and only on eth0 for simplicity. Each test is run for 20 seconds. Table 2 displays the results that were tested for CUBIC, BBR, Illinois, and YeAH congestion control algorithms in Mb/s.

Table 2

CUBIC, BBR, Illinois, and YeAh Bitrate Results with Random Link Loss

Error rate in %	0%	0.01%	0.05%	.1%	.5%	1%	5%	10%	15%	20%
CUBIC	2860	2280	890	482	151	70	11.5	4.4	1.5	.79
BBR	2830	2830	2810	2810	2720	2640	2220	1680	1180	19.80
Illinois	2860	2840	2380	1830	672	107	63.7	96.5	3.2	1.78
YeAH	2860	2230	1930	1860	1310	687	49.6	20	7.1	3.09

Note. Results are displayed in Mb/s

Conclusion: CUBIC performs poorly with radio link loss, which is common in wireless environments. TCP Illinois performed the best at very low radio link loss, at 0.01% random link loss, and still performed decently up to .1% random link loss. BBR performed less efficiently at very low link loss but maintained low RTT and performed excellently with random link loss up to 5%. It still performed efficiently until a steep drop off after 15%, demonstrating that it is the best congestion control for high radio link loss links. TCP YeAH performed better than Illinois after .1% radio link loss and performed twice as well as CUBIC after .05% random packet loss.

Test Link-Loss Tolerant Congestion Control Against a Competing CUBIC flow.

Testing a bottleneck between two competing flows with a 0.1% error rate, 10 ms delay, and a 1 Gb/s rate on Shaper 7, it is found that the radio link-loss aware flow will outcompete the less aggressive link-loss aware flow. At .1% for 60 seconds, a CUBIC flow found 156 Mb/s and sent 1.09 GB. When a competing BBR flow is introduced, CUBIC achieved 70 Mb/s with 500 MB sent, half its non-competing flow amount, while BBR achieved 871Mb/s and sent 6.08 GB of data. It could be seen that BBR is unfair, but testing other link-loss-aware congestion controls shows that they also outcompete non-link-loss-aware congestion controls or less aggressive link-loss-aware congestion controls.

Even when competing against itself, CUBIC loses half its bandwidth due to the congestion signals and random packet loss, limiting its bandwidth. A more interesting event occurs when different radio link loss-tolerant algorithms compete as well.

With a .1% error rate, 1Gb/s, and 10ms delay, BBR found 715Mb/s while Illinois found 233Mb/s after testing for 1 minute, a relative bandwidth of $715/233 = 3.069$. Both these algorithms are link-loss tolerant, but BBR is more aggressive because it only uses latency as a congestion signal. Illinois reduces its congestion window by an eighth when the latency is low,

and packet loss is detected, but it increases its congestion window more aggressively than Westwood, so it also outcompetes Westwood on the link. The results from the January 2025 tests are shown in Table 3, with the left column congestion control algorithm being the left result, the top row congestion control algorithms being the right result, and the data displayed in Mb/s.

It is interesting to note that the CUBIC was also not fair to itself in this situation, as the first starting flow often maintained more bandwidth at the start of the connection. Westwood proved to be fairer to CUBIC flows than a competing CUBIC flow on a radio link loss link, allowing the competing CUBIC flow more bandwidth than either CUBIC flow had when competing with itself. Illinois was not significantly more unfair to a cubic flow than a competing cubic flow, but it was unfair to a Westwood flow.

Conclusion: Radio Link Loss Congestion controls are not fair to each other in the presence of link loss. Their aggression levels range from strongest to weakest: BBR > Illinois >

Table 3

Competing Congestion Controls Over a Random Packet Loss Bottleneck Link

	CUBIC	Westwood	Illinois	BBR
CUBIC	190 v 140			
Westwood	457 v 223	414 v 377		
Illinois	795 v 159	731 v 227	515 v 447	
BBR	851 v 98	861 v 80	715 v 237	493 v 466

Note. Data displayed in Mb/s, competing on a bottleneck with a .1% error rate, and the top row congestion controls correlating with data on the right side of the “v”.

Westwood > CUBIC. Westwood was even more fair and efficient to compete with CUBIC flows in a radio link-loss environment than a competing CUBIC flow.

The Best MPTCP Settings in a Wireless Network

Using Ookla Speedtest on a Google Pixel 7a, it found a 5G connection with 23ms ping, a rate of 639Mbps, jitter of 6 ms, and Packet loss <0.0%; with Wi-Fi 6 it was found to have a 52.8Mb/s, ping at 24 ms, jitter at 67ms, and no data on packet loss. These were used to estimate a plausible wireless simulation environment.

The Shaper 7 bottleneck has a rate of 4Gb/s with a 0.001%, 5ms delay error rate; Shaper 8 simulates a Wifi 6 connection with a .5% error rate, 10ms delay, and 200Mb/s rate; and Shaper 9 simulates a 5G connection with a .01% error rate, 15ms delay, and 800Mb/s rate.

It is first tested that SPTCP-client runs a flow through Shaper 7 to MPTCP-server, which will later be used to create background congestion. By itself, it found 3.76Gb/s flow through Shaper7 after 1 minute.

Then, from the MPTCP-client, an MPTCP flow is tested to the MPTCP-server with congestion created from the SPTCP-client. The same tests are run again as single-path TCP on the best path (the simulated 5G) with congestion from the client. This will simulate a scenario where turning MPTCP on will create more random packet loss signals, which could reduce the overall flow to less than what the single best path available would provide, depending on the selected congestion control. This is particularly true if LIA couple congestion control does, in fact, exist above the congestion control options available in Linux while using MPTCP.

Testing MPTCP with a simulated 5G and Wi-Fi connection with background congestion, a single CUBIC flow at nearly 4 Gb/s on the bottleneck, and each flow ran for 1 minute, the results are presented in Table 4.

Table 4*MPTCP Bitrate on a Congested Network*

Congestion Control	MPTCP bitrate in Mb/s
BIC	839
BBR	815
Scalable	774
Illinois	750
Westwood+	600
YeAH	574
CUBIC	484
H-TCP	460
Veno	374
Highspeed	242
NV	232
Reno	195
Hybla	190
CDG	151
Vegas	113

Conclusions: BIC is a lot more competitive than expected in this environment. It could be expected that its successor would perform better, but in Linux Kernel 6.8, BIC performs well in MPTCP. Westwood+ performed well, and it was shown before that Westwood may be even fairer to CUBIC flows over radio link loss than a competing CUBIC flow. Vegas still proves to be

an excellent low-priority flow. Hybla grows the congestion window very quickly but fails to recover from packet loss events effectively.

Testing again, this time with the lower bandwidth network turned off, which simulated a Wi-Fi 6 network. Table 5 shows the best congestion control for a mobile device in a congested

Table 5

Single-Path TCP Bitrate Over a Congested Network

Congestion Control	SPTCP bitrate in Mb/s
Scalable	760
BIC	759
Illinois	752
BBR	733
H-TCP	661
CUBIC	632
Highspeed	604
Westwood+	589
Reno	435
YeAH	334
Veno	302
Hybla	217
CDG	109
NV	97.2
Vegas	36.4

network using only the 5G (simulated). These are single-path results on an 800Mb/s link

Table 6 shows the improvement of using MPTCP over SPTCP in a congested network.

Table 6

Improvement of MPTCP Over SPTCP in a Congested Network

Congestion Control	MPTCP/SPTCP bandwidth
Vegas	3.10
NV	2.39
YeAH	1.72
CDG	1.39
Veno	1.24
BBR	1.11
BIC	1.11
Westwood+	1.02
Scalable	1.02
Illinois	1.00
Hybla	0.88
CUBIC	0.77
H-TCP	0.70
Reno	0.45
Highspeed	0.40

Note. The data value is found by dividing the Table 4 data by the Table 5 data.

Conclusion: MPTCP is not consistent in improvement over SPTCP depending on the congestion control selected. This may be because LIA does, in fact, work above the congestion control selected, and random packet loss greatly hurts non-radio link loss-aware congestion controls, as each packet loss event is assumed to be on a bottleneck. The congestion controls that suffered the most were unable to differentiate radio link loss, and second, had a slow recovery from a packet loss event.

Depending on the dual link properties, sometimes a bad link will harm the MPTCP flow beyond what the performance of the single best path would provide. For example, a poor Wi-Fi connection and a good 5G connection will slow down to worse than what only the 5G would have provided alone. MPTCP is still a good option, though, as when having both Wi-Fi and 5G on a mobile phone, sometimes data will start to be sent on the worst path in single-path TCP instead of selecting the best path. MPTCP also offers resilience in case of link loss.

BIC and BBR performed best in both tests, slightly improving while using MPTCP, at 111% performance over SPTCP. TCP-Vegas also showed great performance improvement while using MPTCP, showing it may be a great use for Internet of Things devices, as it already has very low bandwidth when competing but also very low RTT. This could be great for Ultra-Reliable-Low-Latency connections. Scalable-TCP had great connection speed in both tests, but due to other research saying it has very unfair RTT unfairness properties, should be avoided. YeAH is a good replacement as it uses Scalable TCP in its fast mode, but it is still outperformed by BBR and BIC.

A reason MPTCP may not be popular is the default congestion control of CUBIC did not perform well with MPTCP turned on. This is plausible, as MPTCP needs both application support to use MPTCP by default, and may then need administration support to use an

appropriate congestion control. It is also plausible that the application can request to use a better congestion control when using MPTCP. BIC performed surprisingly well for being the predecessor to CUBIC.

Chapter Five: Conclusions

This laboratory presents low hardware requirements and can demonstrate many TCP and MPTCP congestion control configurations. It did not seem to recreate RTT unfairness, but it did reliably recreate congestion control differences with simulated radio link-loss packet loss. Further refinement of the methodology can occur in the future by deploying all the virtual machines as containers and automating the deployment of the environment in Proxmox.

TCP and MPTCP share congestion control settings in the Linux kernel 6.8. It was found that MPTCP version one in Linux lacks coupled congestion control or scheduler options. MPTCP was also found to not be perfectly fair to other flows, though found to be very fair with Jain's Fairness Index, and has inconsistent delivery of service, finding .75 to 1.5 relative flows worth of bandwidth, often 1.25 flows worth of bandwidth when competing in an equal environment, showing a slight advantage in resource allocation. In many of these tests, the competing single-path TCP flow against MPTCP is set to CUBIC, the default congestion control for Linux, Windows, and Apple devices.

TCP-LP (Low Priority) was found to be dysfunctional and unable to not interfere with bandwidth from competing flows as intended.

It was found that most radio-link loss-tolerant congestion control mechanisms can act unfairly against a less aggressive TCP flow when competing over an error-prone link. BBR unfairly outcompetes Illinois, Westwood, and CUBIC; Illinois outcompetes CUBIC and Westwood, and Westwood was shown to be fairer to CUBIC than even other competing CUBIC flows in a random link loss environment. It seems that differences in handling radio link packet loss can create unfairness.

While using MPTCP in a simulated environment for a cellphone device, one 5G connection and one Wi-Fi 6 connection, the best congestion control for throughput was found to be BIC and BBR, followed by Scalable-TCP and TCP-Illinois. It was found that in many cases, a single good TCP flow on a 5G connection may outperform an MPTCP connection using both 5G and the Wi-Fi 6 connection, but this is also dependent on the congestion control used, how asymmetrical the links are, and how poorly one of the links connections is. CDG congestion control did not perform well in either scenario.

CUBIC congestion control is not radio-link loss tolerant, which makes it improper to deliver service to wireless devices. Many options are available in the Linux kernel as alternatives. Despite performing very well, TCP-Scalable is not recommended as it has been shown to have severe RTT unfairness in other studies. TCP-Westwood is a conservative solution, performs well in both SPTCP and MPTCP, and has been shown to be fair to TCP-CUBIC flows in the presence of packet loss. BBR showed good performance and has maintained low RTT during flows. TCP-BIC performed surprisingly well despite not being radio link loss aware, and was the predecessor to CUBIC. TCP-Illinois was expected to have better performance when using the MPTCP minRTT scheduler, but failed to produce the anticipated results.

The best set-up for MPTCP in a simulated wireless network seems to be BBR and BIC for resilience, Westwood for fairness, and Vegas for low RTT, low priority flows. In some instances, the single-path 5G network will have better performance than 5G plus Wi-Fi 6 networking with MPTCP. The benefits of reliability and smooth stream handoff in MPTCP may outweigh the possible bandwidth loss when using MPTCP on uneven networks. It also needs to be considered that whatever congestion control is selected, if MPTCP falls back onto regular TCP, that congestion control will still be used.

To use MPTCP, applications must be designed to use it natively or forced to use it with outside controls, such as using "mptcpize," which was used in this study. Unlike TCP or MPTCP, applications can use QUIC or MPQUIC without requiring a kernel upgrade and may be able to select their own congestion controls and schedulers on the receiving devices.

While QUIC and MPQUIC may be more difficult to test, they use the same congestion controls and foundation as TCP and MPTCP. They are also more easily deployed, where TCP needs a kernel upgrade, and QUIC does not. QUIC is currently being deployed as the standard internet traffic protocol in HTTP/3. Though this study does not run experiments using QUIC and MPQUIC, these congestion controls and multipath configurations can be deployed with the latest internet protocols. MPQUIC is in internet draft and will likely be deployed for many mobile device applications. Efficient and fair use of multipath internet protocols will be crucial for delivering services to wireless devices now and in the future.

References

- Alizadeh, M., Greenberg, A., Maltz, D. A., Padhye, J., Patel, P., Prabhakar, B., Sengupta, S., and Sridharan, M. (2010). *Data center tcp (dctcp)*. *Proceedings of the ACM SIGCOMM 2010 Conference*, pages 63–74.
- Allman, M., Paxson, V., and Blanton, E. (2009). RFC 5681 Tcp congestion control. IETF.
<https://rfc-editor.org/rfc/rfc5681>
- Baiocchi, A., Castellani, A. P., Vacirca, F., et al. (2007). *Yeah-tcp: yet another highspeed tcp*. *Proceedings. PFLDnet, volume 7*, pages 37–42.
- Bishop, M. (2022). RFC 9114 HTTP/3. IETF. <https://rfc-editor.org/rfc/rfc9114>
- Braden, R. (1989). RFC1122 Requirements for internet hosts-communication layers. IETF.
<https://rfc-editor.org/rfc/rfc1122>
- Brakmo, L. (2015). Tcp-nv: An update to tcp-vegas. Google Docs.
https://docs.google.com/document/d/1o-53jbO_xH-m9g2YCgjaf5bK8vePjWP6Mk0rYiRLK-U/edit?usp=sharing
- Brakmo, L. S. and Peterson, L. L. (1995). Tcp vegas: End to end congestion avoidance on a global internet. *IEEE Journal on selected Areas in communications*, 13(8):1465–1480.
- Caini, C. and Firrincieli, R. (2004). Tcp hybla: a tcp enhancement for heterogeneous networks. *International journal of satellite communications and networking*, 22(5):547–566.
- Cardwell, N., Cheng, Y., Gunn, C. S., Yeganeh, S. H., and Jacobson, V. (2017). Bbr: congestion-based congestion control. *Communications of the ACM*, 60(2):58–66.
- Chaudhary, S., Maity, M., Chakraborty, S., and Shukla, N. (2024). A dataset for analyzing streaming media performance over http/3 browsers. *Advances in Neural Information Processing Systems*, 36.

Cloudflare (2024 June 26). *Cloudflare radar adoption usage in United States*.

<https://radar.cloudflare.com/adoption-and-usage/US>

Dell'Aera, A., Grieco, L. A., and Mascolo, S. (2004). Linux 2.4 implementation of westwood+ tcp with rate-halving: A performance evaluation over the internet. In 2004 IEEE International Conference on Communications (IEEE Cat. No. 04CH37577), volume 4, pages 2092–2096. IEEE.

Deng, Z., Liu, Y., Liu, J., Argyriou, A., and Liu, D. (2024). Bbr-based and fairness guaranteed congestion control and packet scheduling for mpquic over heterogeneous networks. *Computer Communications*, 224:213–224.

Drucker, R., Baraskar, G., Balasubramanian, A., and Gandhi, A. (2024). Bbr vs. bbrv2: A performance evaluation. In 2024 16th International Conference on COMMunication Systems & NETWORKS (COMSNETS), pages 379–387. IEEE.

Eddy, W. (2022). RFC 9293 Transmission control protocol (TCP). IETF. <https://www.rfc-editor.org/rfc/rfc9293.html>

Enotiadis, A. (2023). Nanog: 4760 the new, encrypted protocol stack how to deal with it. Youtube. <https://youtu.be/A3gFqi3eUC8?si=QsylBUUcsOppoSBO>

Floyd, S. (2003). RFC3649 Highspeed tcp for large congestion windows. IETF. <https://www.rfc-editor.org/rfc/rfc3649.html>

Ford, A., Raiciu, C., Handley, M., Bonaventure, O., and Paasch, C. (2020). RFC 8684 Tcp extensions for multipath operation with multiple addresses. IETF. <https://www.rfc-editor.org/rfc/rfc8684.html>

- Ford, A., Raiciu, C., Handley, M. J., and Bonaventure, O. (2013). RFC 6824 TCP Extensions for Multipath Operation with Multiple Addresses. IETF. <https://www.rfc-editor.org/rfc/rfc6824>
- Fu, C. P. and Liew, S. C. (2003). Tcp veno: Tcp enhancement for transmission over wireless access networks. *IEEE Journal on selected areas in communications*, 21(2):216–228.
- Gomez, J., Kfoury, E. F., Crichigno, J., and Srivastava, G. (2024). Evaluating tcp bbrv3 performance in wired broadband networks. *Computer Communications*, 222:198–208.
- Ha, S. and Rhee, I. (2011). Taming the elephants: New tcp slow start. *Computer Networks*, 55(9):2092–2110.
- Ha, S., Rhee, I., and Xu, L. (2008). CUBIC: a new tcp-friendly high-speed tcp variant. *ACM SIGOPS operating systems review*, 42(5):64–74.
- Hayes, D. A. and Armitage, G. (2011). Revisiting tcp congestion control using delay gradients. In *International Conference on Research in Networking*, pages 328–341. Springer.
- Iyengar, J. and Thomson, M. (2021). RFC 9000: Quic: A udp-based multiplexed and secure transport. IETF. <https://www.rfc-editor.org/rfc/rfc9000>
- Jacobson, V. (1988). Congestion avoidance and control. *ACM SIGCOMM computer communication review*, 18(4):314–329.
- Jain, R., Chiu, D.-M., & Hawe, W. (1984). *A quantitative measure of fairness and discrimination for resource allocation in shared computer systems* (Tech. Rep. TR-301). Digital Equipment Corporation.
- Jowkarishasaltaneh, F. and But, J. (2022). An analysis of mptcp congestion control. In *Telecom*, volume 3, pages 581–609. MDPI.

- Kato, T., Diwakar, A., Yamamoto, R., Ohzahata, S., and Suzuki, N. (2019). Experimental analysis of mptcp congestion control algorithms; lia, olia and balia. In 8th International Conference on Theory and Practice in Modern Computing (TPMC 2019), pages 135–142.
- Kelly, T. (2003). Scalable tcp: Improving performance in highspeed wide area networks. ACM SIGCOMM computer communication Review, 33(2):83–91.
- Khalili, R., Gast, N., Popovic, M., and Le Boudec, J.-Y. (2013). Mptcp is not pareto-optimal: Performance issues and a possible solution. IEEE/ACM Transactions On Networking, 21(5):1651–1665.
- Kharat Prashant, K. (2019). *Modified QUIC Protocol with Congestion Control for Improved Network Performance*. PhD thesis, National Institute of Technology Karnataka, Surathkal.
- Kuzmanovic, A., & Knightly, E. (2003). Tcp-lp: A distributed algorithm for low priority data transfer. IEEE INFOCOM 2003. Twenty-second Annual Joint Conference of the IEEE Computer and Communications Societies (IEEE Cat. No.03CH37428), 3, 1691–1701 vol.3. <https://doi.org/10.1109/INFCOM.2003.1209192>
- Langley, A., Riddoch, A., Wilk, A., Vicente, A., Krasic, C., Zhang, D., Yang, F., Kouranov, F., Swett, I., Iyengar, J., et al. (2017). *The quic transport protocol: Design and internet scale deployment*. In *Proceedings of the conference of the ACM special interest group on data communication*, pages 183–196.
- Leith, D. and Shorten, R. (2004). *H-tcp: Tcp for high-speed and long-distance networks*. In *Proceedings of PFLDnet, volume 2004*. Citeseer.

Linux Kernel Development (2024, October 18). Linux kernel stable tree.

<https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/tree/?h=v6.10.12>.

Linux MPTCP Upstream Project (2024, October 18). Linux mptcp upstream project changelog.

https://github.com/multipath-tcp/mptcp_net-next/wiki#changelog.

Liu, S., Başar, T., and Srikant, R. (2006). *Tcp-illinois: A loss and delay-based congestion control algorithm for high-speed networks*. In *Proceedings of the 1st international conference on Performance evaluation methodologies and tools*, pages 55–es.

Liu, Y., Ma, Y., Coninck, Q. D., Bonaventure, O., Huitema, C., and KÄhlewind, M. (2024). Multipath Extension for QUIC. Internet-Draft draft-ietf-quic-multipath-10, Internet Engineering Task Force. Work in Progress.

Łuczak, Ł. P., Ignaciuk, P., and Morawski, M. (2023). Evaluating mptcp congestion control algorithms: Implications for streaming in open internet. *Future Internet*, 15(10):328.

Mahmud, I., Lubna, T., Song, Y.-J., and Cho, Y.-Z. (2020). Coupled multipath bbr (c mpbbr): An efficient congestion control algorithm for multipath tcp. *IEEE access*, 8:165497–165511.

Mascolo, S., Casetti, C., Gerla, M., Sanadidi, M. Y., and Wang, R. (2001). *Tcp westwood: Bandwidth estimation for enhanced transport over wireless links*. In *Proceedings of the 7th annual international conference on Mobile computing and networking*, pages 287–297.

MPTCP Dev Team (2024, September 30). Multipath tcp for linux faq.

<https://www.mptcp.dev/faq.html#mptcpv0-vs-mptcpv1>

onemarcfifty. (2021). *GitHub - onemarcfifty/proxmox-network: Scripts and Stuff around the Proxmox Network*. GitHub. <https://github.com/onemarcfifty/Proxmox-network>

- Paasch, C., Barre, S., et al. (2021). Multipath TCP in the Linux Kernel. <https://multipath-tcp.org/pmwiki.php/Users/ConfigureMPTCP>
- Postel, J. (1981). RFC 793 Transmission Control Protocol. IETF. <https://www.rfc-editor.org/rfc/rfc793>
- Raiciu, C., Handley, M. J., and Wischik, D. (2011). Coupled Congestion Control for Multi path Transport Protocols. RFC 6356. IETF. <https://www.rfc-editor.org/rfc/rfc6356>
- Sengupta, J., Kosek, M., Fries, J., Ferlin-Reiter, S., and Bajpai, V. (2024). On cross-layer interactions of quic, encrypted dns and http/3: design, evaluation and dataset. IEEE Transactions on Network and Service Management.
- Shu, S., Yang, W., Pan, J., and Cai, L. (2023). *A multipath extension to the quic module for ns-3*. In *Proceedings of the 2023 Workshop on ns-3*, pages 86–93.
- Stoica, I. (2005). A comparative analysis of tcp tahoe, reno, new-reno, sack and vegas.
- Trevisan, M., Giordano, D., Drago, I., and Khatouni, A. S. (2021). Measuring http/3: Adoption and performance. In 2021 19th Mediterranean Communication and Computer Networking Conference (MedComNet), pages 1–8. IEEE.
- Viernickel, T., Froemmgen, A., Rizk, A., Koldehofe, B., and Steinmetz, R. (2018). Multipath quic: A deployable multipath transport protocol. In 2018 IEEE International Conference on Communications (ICC), pages 1–7.
- Vu, M. H., TT Nguyen, G., Tran, H. D., Nguyen, T. T., Do, P. T., Nguyen, P. L., and Nguyen, K. (2022). *An empirical study of mpquic schedulers in mobile wireless networks*. In *Proceedings of the 11th International Symposium on Information and Communication Technology*, pages 193–200.

- Wei, W., Xue, K., Han, J., Xing, Y., Wei, D. S., and Hong, P. (2020). Bbr-based congestion control and packet scheduling for bottleneck fairness considered multipath tcp in heterogeneous wireless networks. *IEEE Transactions on Vehicular Technology*, 70(1):914–927.
- Xu, L., Ha, S., Rhee, I., and Goel, V. (2023). Rfc 9438: CUBIC for fast and long-distance networks. IETF. <https://www.rfc-editor.org/rfc/rfc9438>
- Xu, L., Harfoush, K., and Rhee, I. (2004). Binary increase congestion control (bic) for fast long-distance networks. In *IEEE INFOCOM 2004*, volume 4, pages 2514–2524. IEEE.
- Zheng, Z., Ma, Y., Liu, Y., Yang, F., Li, Z., Zhang, Y., Zhang, J., Shi, W., Chen, W., Li, D., et al. (2021, August). *Xlink: Qoe-driven multi-path quic transport in large-scale video services*. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, pages. 418-432.

Appendix A: Proxmox Install

Proxmox is a Linux-based virtual environment capable of running virtual machines and containers. First, the lab installed a Proxmox virtual environment on an AMD FX-6300 desktop with 8 GB DDR3 RAM and a 2TB hard drive. Proxmox was downloaded from the website [<https://www.Proxmox.com/en/downloads>], and Rufus was used to create a bootable USB to install. The initial Proxmox installation on the desktop was difficult because the video driver loading would get stuck before Proxmox could be installed. This was due to using a particular Nvidia graphics card for the display. The install was stuck on “create LV” at 3%. In order to proceed passed this error, run the terminal install, edit the options (click “e” for edit when selecting options), and change line “vga=788” to “nomodeset.” This allowed the installation to proceed with the Nvidia video card. After setting a static IP address, we can log in to the Proxmox server using a browser, in this case with "192.168.0.10:8006."

Upon installing Proxmox, the Proxmox update repository needs to be changed to non-subscription, and the "no valid subscription" nag box can be removed. To switch to no subscription repository, comment out “/etc/apt/source.list.d/ceph.list” and “/etc/apt/source.list.d/pve-enterprise.list”, both files under enterprise. Open “/etc/apt/source.list” in a text editor and ensure “HTTP://download.Proxmox.com/debian/pve” is included. Open PVE shell and run “apt update” to update the Proxmox server.

Adding ISO disks to the Proxmox server entailed going to local (pve) volume, selecting ISO Images, and selecting "Download from URL." An Ubuntu 24.04 iso was used.

Appendix B: Network Setup

Many network settings were inspired by Youtuber OneMarcFifty's "Proxmox-network" (onemarcfifty, 2021). Proxmox requires virtual bridges, or switches, to establish network connections.

There is an internet bridge to the real internet, vmbr0, which is required to download apps and updates for VMs, including `iperf3` and `mptcpize`. The following bridges and settings were used:

Left bridge vmbr6 - 192.168.56.0/24

shaper bridges:

vmbr7 - 10.7.0.0/24

vmbr8 - 10.8.0.0/24

vmbr9 - 10.9.0.0/24

vmbr10 - 10.10.0.0/24

To install the shapers, Debian containers need to be installed. These shapers need to be configured to allow IP forwarding, allow NAT, and have routing tables to route traffic correctly.

In Proxmox Terminal, run `"pveam update"` to update the list, then `"pveam available -section system"` to see available options. To download the container, run `"pveam download local CONTAINER-NAME"` where `CONTAINER-NAME` is the Linux version to install. In this laboratory, we used Debian 10, which used the Linux 6.8 kernel.

CT templates will be stored in local(pve) with default settings under CT Templates.

Linux Shapers Setup

Create Linux containers with Proxmox "Create CT"

This will allow for a root password and ask for a container template. We use Debian 10. CPU is set to 1, Memory is set to 128MB. A single network interface is asked, and it is important to enter a static IP; this will be on the left side of our network. No DNS is needed. Afterward, before starting, a second Network device can be attached, and this will be the right side of the network. The shapers will be logged in as the root user, so the commands will not need `sudo`. These are only connected to the private network and do not need updates

Check Linux Kernel version `"uname -r"`

Check OS `"lsb_release -a"`

Check Linux Kernel and OS `"cat /proc/version"`

Configure IP Forwarding

Turn Debian containers into routers with the command `"sudo sysctl net.ipv4.ip_forward=1"` Check with `"sysctl net.ipv4.ip_forward"` if the answer is 0, it is not configured as a router. If 1, it is configured as a router. Routing tables need to be updated manually.

To turn this permanent after reboots, edit file `/etc/sysctl.conf`, and change the line with the value `"net.ipv4.ip_foward"`, uncomment it, and set it equal to `"1"`. Also, be sure that TCP timestamps are enabled by setting `"net.ipv4.tcp_timestamps = 1"`. This is important for the TCP-LP test, as the Low Priority TCP should rely on timestamps as a pre-emptive signal for congestion control

Configure Network Address Translation

Most issues with getting the Debian containers to act as routers are typically due to NAT not being configured or the routing configuration not being set correctly. Check the part forwarding rules with "**sudo iptables -s**". This needs to be configured correctly to allow port forwarding, also called NAT.

The commands should be:

```
iptables -P INPUT ACCEPT
iptables -P OUTPUT ACCEPT
iptables -P FORWARD ACCEPT
iptables -A FORWARD -j ACCEPT
```

See configured iptables with "**iptables -s**"

Make this configuration permanent with "**mkdir /etc/iptables | iptables-save | tee /etc/iptables/rules.v4**"

Configure NAT (port-forwarding) with "**iptables -t nat -A POSTROUTING -j MASQUERADE**".

This needs to be run at startup for the router to work correctly. In order for this to run at startup, we use the `/etc/rc.local` file. This file must be created, changed to allow executable, and have the "**iptables -t nat -A POSTROUTING -j MASQUERADE**" written in. Figure A1 is an example of the `/etc/rc.local` file from Shaper8. This will slow down the restart speed of the container, but will set NAT to be added after each restart.

Figure A1

cat /etc/rc.local on Shaper8

```
root@Shaper8 ~# cat /etc/rc.local
#!/bin/bash

iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE

exit 0
root@Shaper8 ~#
```

Configure Routing Tables.

Show static routes with "ip route show".

Add correct routes with "ip route add {NETWORK} via {IP} dev {DEVICE}" The network should have the correct subnet, the {IP} needs to be the destination router's IP address for that network. Figure A2 is Shaper8 after running ip route show, showing the required routing configurations. Figure A3 is the ip route show on Shaper9, and Figure A4 is the ip route show on Shaper7, the traffic shaper used for the bottleneck to create congestion. If the traffic is not flowing through the traffic shapers (you can test this with the ping command), please ensure the NAT and routing tables are correctly configured.

Figure A2

ip route show on Shaper8

```
root@Shaper8 ~# ip route show
default via 10.7.0.1 dev eth0 onlink
10.7.0.0/24 dev eth0 proto kernel scope link src 10.7.0.2
10.8.0.0/24 dev eth1 proto kernel scope link src 10.8.0.1
root@Shaper8 ~#
```

Figure A3

ip route show on Shaper9

```
root@Shaper9-3 ~# ip route show
default via 10.10.0.1 dev eth0 onlink
10.9.0.0/24 dev eth1 proto kernel scope link src 10.9.0.1
10.10.0.0/24 dev eth0 proto kernel scope link src 10.10.0.2
root@Shaper9-3 ~#
```

Figure A4

ip route show on Shaper7

```
root@Shaper7 ~# ip route show
default via 192.168.56.1 dev eth0 onlink
10.7.0.0/24 dev eth1 proto kernel scope link src 10.7.0.1
10.9.0.0/24 dev eth2 scope link
10.10.0.0/24 dev eth2 proto kernel scope link src 10.10.0.1
192.168.56.0/24 dev eth0 proto kernel scope link src 192.168.56.7
root@Shaper7 ~#
```

To make these configurations permanent, either add the correct `ip route` commands to the `/etc/rc.local` file, or add the `ip route` commands to the `/etc/network/interfaces`. In both cases this will re-run the commands on each startup. Figure A5, Figure A6, and Figure A7 present screenshots of the configuration settings in the `/etc/network/interfaces` file on Shaper8, Shaper9, and Shaper7 respectively.

Figure A5

cat /etc/network/interfaces on Shaper8

```
# UNCONFIGURED INTERFACES
# remove the above line if you edit this file

auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
    address 10.7.0.2/24
    gateway 10.7.0.1
    up ip route del 10.7.0.0/24 via 10.7.0.1 dev eth0
    up ip route add 10.7.0.0/24 via 10.7.0.1 dev eth0

allow-hotplug eth1
auto eth1
iface eth1 inet static
    address 10.8.0.1/24
    up ip route del 10.8.0.0/24 via 10.8.0.1 dev eth1
    up ip route add 10.8.0.0/24 via 10.8.0.1 dev eth1

~
~
~
"/etc/network/interfaces" 20L, 456C
```

Figure A6

cat /etc/network/interfaces on Shaper9

```
root@Shaper9-3 ~# cat /etc/network/interfaces
# UNCONFIGURED INTERFACES
# remove the above line if you edit this file

auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
    address 10.10.0.2/24
    gateway 10.10.0.1
    up ip route del 10.10.0.0/24 via 10.10.0.1 dev eth0
    up ip route add 10.10.0.0/24 via 10.10.0.1 dev eth0

allow-hotplug eth1
auto eth1
iface eth1 inet static
    address 10.9.0.1/24
    up ip route del 10.9.0.0/24 via 10.9.0.1 dev eth1
    up ip route add 10.9.0.0/24 via 10.9.0.1 dev eth1

root@Shaper9-3 ~#
```

Figure A7

cat /etc/network/interfaces on Shaper7

```
root@Shaper7 ~# cat /etc/network/interfaces
# UNCONFIGURED INTERFACES
# remove the above line if you edit this file

auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
    address 192.168.56.7/24
    gateway 192.168.56.1
    ip route del default via 192.168.56.1 dev eth0
    ip route add default via 192.168.56.1 dev eth0

allow-hotplug eth1
auto eth1
iface eth1 inet static
    address 10.7.0.1/24
    ip route del 10.7.0.0/24 via 10.7.0.1 dev eth1
    ip route add 10.7.0.0/24 via 10.7.0.1 dev eth1
    ip route del 10.8.0.0/24 via 10.7.0.2 dev eth1
    ip route add 10.8.0.0/24 via 10.7.0.2 dev eth1

iface eth0 inet6 dhcp

auto eth2
iface eth2 inet static
    address 10.10.0.1/24
    ip route del 10.10.0.0/24 via 10.10.0.1 dev eth2
    ip route add 10.10.0.0/24 via 10.10.0.1 dev eth2
    ip route del 10.9.0.0/24 via 10.10.0.2 dev eth2
    ip route add 10.9.0.0/24 via 10.10.0.2 dev eth2

root@Shaper7 ~#
```

Appendix C: Traffic Shaper Commands

The TC command is used to apply network emulation on the outgoing traffic on the shapers. The first command is used to apply the traffic emulation, such as: `“tc qdisc add dev eth1 root netem loss random .1% delay 20ms rate 100mbps”`.

This translates to traffic control, queueing discipline, adding network emulation on device eth1, applying a random packet loss of .1%, a delay of 20ms to outgoing packets, and a rate of 100 megabytes per second. The rate is in bytes, while `iperf` measures in bits. So, 100mbps will yield an 800 Mbit/s in bandwidth. To specify megabits per second in `tc`, use “mbit” instead of “mbps.” For example: `“rate 100mbps”` would now be `“rate 800mbit”`.

After first adding the `tc` settings to an outgoing traffic port, it can be modified with a “change” instead of an “add”, or deleted with “delete”. For changing the network emulation rules on outgoing traffic, for example: `“tc qdisc change dev eth1 root netem loss random .5% delay 20ms rate 200mbit”`

To add a delay variation, you can add random variation like such: `“tc qdisc change dev eth0 root netem delay 20ms 3ms”` which randomly variates delay by 3ms, or it can variate with a normal distribution like such: `“tc qdisc change dev eth0 root netem delay 20ms 3ms distribution normal”`.

The random `loss` can be changed to `corrupt` or `duplicate` in the command to change from dropped packets to corrupted packets or duplicated packets respectively.

To delete all the rules with “eth1” being the egress port you can use:

```
“tc qdisc del dev eth1 root”
```

To see applied rules with “eth1” being the egress port you can use:

```
“tc qdisc show dev eth1”
```


Appendix D: Linux VM Set Up

Linux ISO images can be downloaded from their sources, go to local (pve) > ISO Images
> Download from URL.

Check Linux Kernel version with the command `"uname -r"`

The VMs used in this lab are Ubuntu 22 and Ubuntu 24, but the minimal requirements are only to have a Linux 6.8 kernel machine (as a container or VM) and to install `iperf3` and `mptcpize`.

The `iperf3` command can change the congestion control for tests, but many of the TCP congestion controls need to be activated first. This can be done with the command `"sudo sysctl net.ipv4.tcp_congestion_control=reno"` (using `reno` for example). Each congestion control will need to be activated once before it can be used in the `iperf` command

Left VM "MPTCP-server" is Linux 6.8.0-40-generic on a 22.04 Ubuntu distribution. It has `iperf3` and `mptcpize` installed. It runs `iperf3` in server mode on two different ports.

Right VM "MPTCP-client" is Linux 6.8.0.41-generic, and a 24.04 Ubuntu distribution. It has `iperf3` and `mptcpize` installed.

The Middle VM "SPTCP-client" is a clone of "MPTCP-client" and is used to create congestion to compete with MPTCP-client.

Appendix E: MPTCP Settings

According to the official MPTCP version zero documentation (C. Paasch, S. Barre, et al., 2021) to get MPTCP version with the command: `dmesg | grep MPTCP`. To see if mptcp is enabled run `sysctl net.mptcp.enabled` and this should return `net.mptcp.enabled = 1` by default.

Change congestion control with `sysctl net.ipv4.tcp_congestion_control=` This is the same location as the single path congestion controls. This is how we change the congestion control in the lab. The MPTCP version 0 options for coupled congestion control were LIA, OLIA, wVegas, and BALIA. These do not appear to be available in MPTCP version 1 in the Linux Kernel implementation. Most congestion controls are not available with `sysctl net.ipv4.tcp_available_congestion_control` until having been activated at least once.

`sysctl net.mptcp.mptcp_path_manager=` can be used to change path manager, which will not be used in this lab. This controls how new subflows are established.

The scheduler should be changed with `sysctl net.mptcp._scheduler` but this does not appear to contain any options in MPTCP version 1 in Linux. These options were "default" or "minRTT", Round Robin, and Redundant in the MPTCP version 0 (C. Paasch, S. Barre, et al., 2021)

Restarting the VMs does not save the congestion controls; they have to be reactivated with `sysctl net.ipv4.tcp_congestion_control=` after each restart. These commands can be put in the `/etc/rc.local` file to be run at startup if needed.

Appendix F: `iperf3` Commands

`iperf3` settings:

`-C` can set congestion control

`-p` sets port

`-J` outputs a json files

`-t` sets time

`-c` sets the client

`-s` sets the server

`"mptcpize run iperf3 ..."` sets `iperf3` into MPTCP mode

MPTCP-server needs to run `iperf3` in server mode on at least two ports, and also needs to be run through `mptcpize` to utilize MPTCP. The first port can be open with `"mptcpize run iperf3 -s"` which will open `iperf3` on port 5201. A second port will need to be run in another terminal window with `"mptcpize run iperf3 -s -p 5202"` to open a second `iperf3` on port 5202.

The SPTCP-client will run `iperf` with such a command as `"iperf3 -c 192.168.56.11 -t 30 -C CUBIC"` which will set `iperf3` to run for 30 seconds using congestion control CUBIC. Congestion controls need to be activated with `"sysctl net.ipv4.tcp_congestion_control=bbr"` after each restart before they can be used in `iperf3`.

The MPTCP-client will run `iperf3` in mptcp mode with a command such as `"mptcpize run iperf3 -c 192.168.56.11 -t 600 -C bbr -p 5202"` which will run `iperf3` in mptcp mode for 600 seconds using BBR congestion control onto port 5202. SPTCP-client and MPTCP-client must use two different ports at a time in order to send traffic to MPTCP-server at the same time.